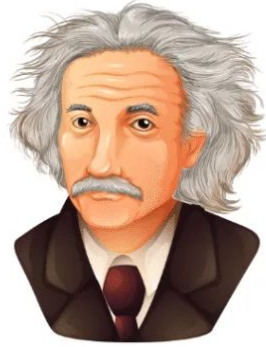


Chapter 4: Machine Foundation (BCT)

By Er. Ukesh Thapa

Syllabus

- ❖ Foundations and Four pillars of Machine Learning
 - ❖ Review of Mathematics for Machine Learning
 - ❖ Continuation Optimization; Unconstraint Optimization, Constraint Optimization, Convex Optimization
 - ❖ Review of Machine Learning Pipeline
 - Model Development: Generative vs. Discriminative:
 - Learning Algorithm
 - Capacity, Overfitting and Underfitting
 - Hyperparameters and Validation Sets
 - Estimators, Bias and Variance
 - Review of MLE & MAP and Cross Entropy
 - ❖ Supervised Learning Algorithm: Decision Tree and Support Vector Machine
 - ❖ Unsupervised Learning Algorithm: t-SNE
 - ❖ Semi Supervised and Reinforcement Learning
 - ❖ Model Evaluation: Energy-based Indicators
- Time: 10 hrs Marks: 12**



It is not that I'm so
smart. But I stay with
the questions much
longer.

Albert Einstein

LEARNING FROM EXAMPLES

- ❖ An agent is learning if it improves its performance after making observations about the world.

Machine learning

- ❖ Field of study that gives computers the ability to learn without being explicitly programmed, by **Arthur Samuel** in 1959
- ❖ A computer program is said to learn from experience (**E**) with respect to some class of tasks (**T**) and performance measure (**P**), if its performance at tasks in **T**, as measured by **P**, improves with experience **E**. by **Tom Mitchell** in 1997

Why do Machines Need to Learn?

- ❖ Some Tasks are Too Complex to Define with Rules
- ❖ The Environment is Always Changing
- ❖ Personalization at Scale
- ❖ Discovering Hidden Patterns

While Traditional programming uses Data and Rules to provide a Answers

Factors How Agents Learns?

- ❖ Which component is to be improved?
 - Machine Learning can be applied to fix or optimize specific parts of this pipeline.
 - You don't always "learn the whole agent"; often, you just learn one specific sub-task.
- ❖ What prior knowledge does the agent have?
 - This factor determines how much the agent has to learn from scratch.
 - This is often described as the trade-off between Nature (hard-coded knowledge) and Nurture (learned knowledge).
- ❖ What data and feedback is available?
 - The "Feedback" is the information the environment gives back to the agent after it processes data.

Forms of Learning

Supervised Learning

- ❖ The agent observes some example input-output pairs (x, y) and learns a function (f) that maps inputs to outputs ($f(x) \approx y$) so it can predict the output for new, unseen inputs.
- ❖ Learning with an answer key.
- ❖ The data is Labeled.
- ❖ **Applications:**
 - Face Recognition
 - Spam Filters
 - Medical Diagnosis
 - Stock Price Prediction

Supervised Learning Approaches

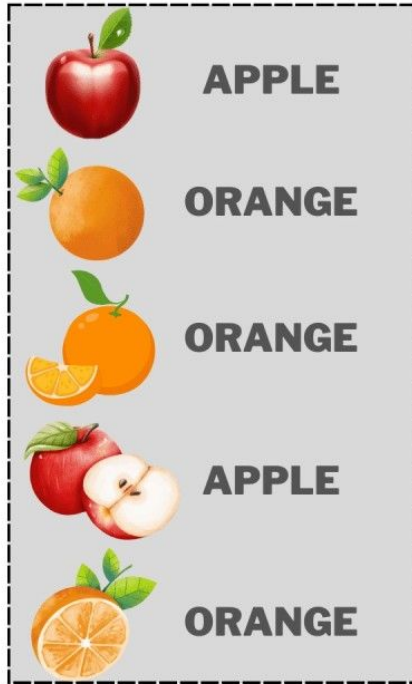
❖ Regression Approaches

- **Linear Regression:** The "straight line" approach. Simplest tool.
- **Polynomial Regression:** For curved patterns.

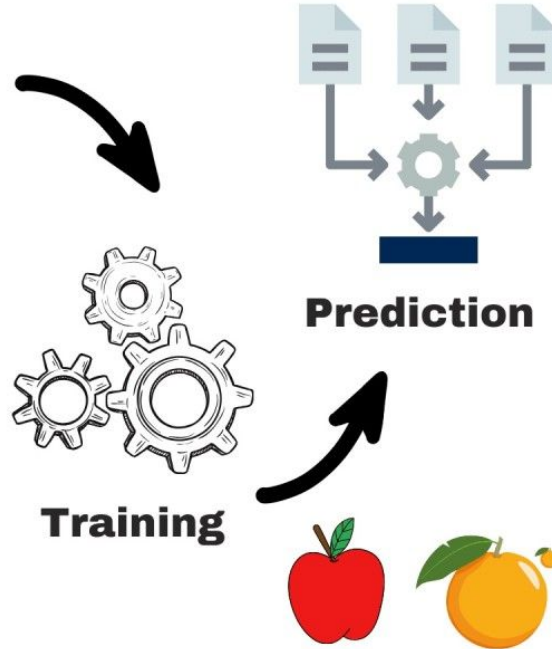
❖ Classification Approaches

- **Logistic Regression:** Despite the name, it predicts probabilities (0 to 1) to classify data (e.g., Spam vs. Not Spam).
- **Decision Trees:** A flowchart-like structure where each "node" asks a question to split the data.
- **Naive Bayes:** Based on probability theory (Bayes' Theorem). It assumes features are independent. Great for text classification.
- **Support Vector Machines (SVM):** Finds the widest possible boundary ("street") between two classes.
- **K-Nearest Neighbors (KNN):** The "peer pressure" algorithm. If your 5 nearest neighbors are Red, you are probably Red.

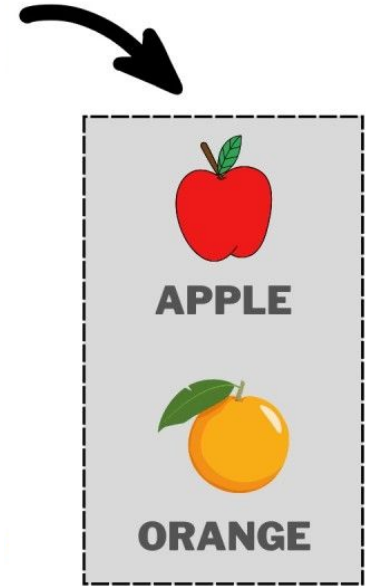
Supervised Learning



Labeled Dataset



Test Data

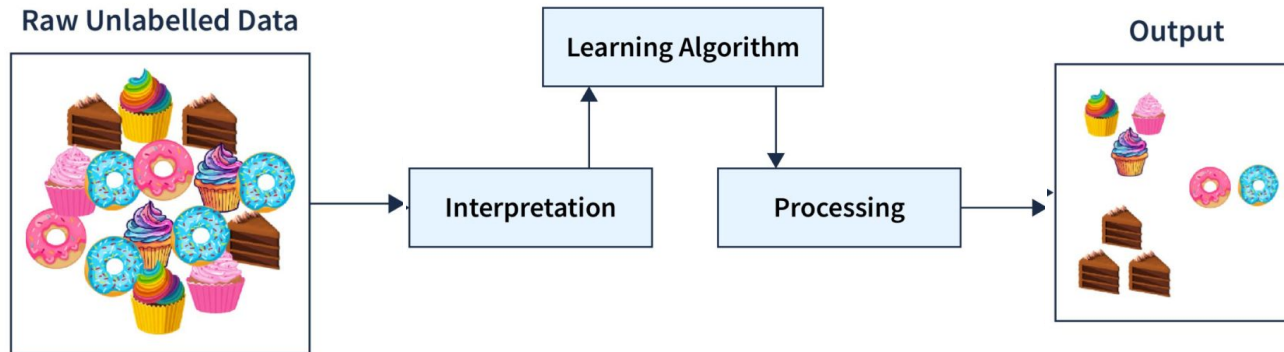


Unsupervised Learning

- ❖ The agent learns patterns in the input x without any specific output values y supplied.
- ❖ It seeks to model the underlying structure or distribution of the data.
- ❖ The agent groups similar things together based on their inherent properties.
- ❖ The data is Unlabeled.
- ❖ **Applications:**
 - Customer Segmentation
 - Recommender Systems
 - Anomaly Detection

Approaches for Unsupervised Learning

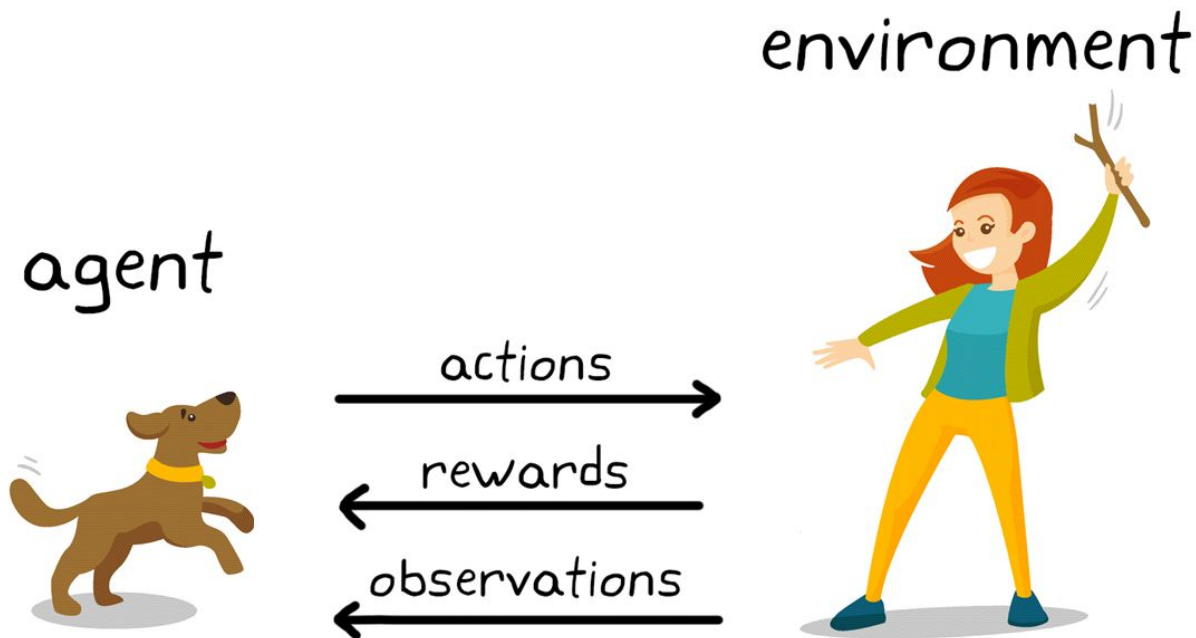
- ❖ **Clustering Approaches (Grouping)**
 - K-Means
 - Hierarchical
- ❖ **Dimensionality Reduction Approaches (Simplifying)**
 - PCA (Principal Component Analysis)



Reinforcement Learning

- ❖ An agent learns a policy to maximize expected cumulative reward by interacting with an environment. It learns through trial and error.
- ❖ Good actions get a (positive reward), bad actions get a (negative penalty).
- ❖ Feedback for the better performance
- ❖ **Applications:**
 - Game Playing: AlphaGo (Go), OpenAI Five (Dota 2), Chess engines.
 - Robotics: A robot dog learning to walk (falling = penalty, moving forward = reward).
 - Self-Driving Cars: Navigating traffic without crashing.
- ❖ **Approaches**
 - **Q-Learning:** Learning the value of taking a certain action in a certain state.
 - **Deep Q-Networks (DQN):** Using Neural Networks to handle complex states.
 - **Policy Gradients:** Directly optimizing the agent's behavior strategy.

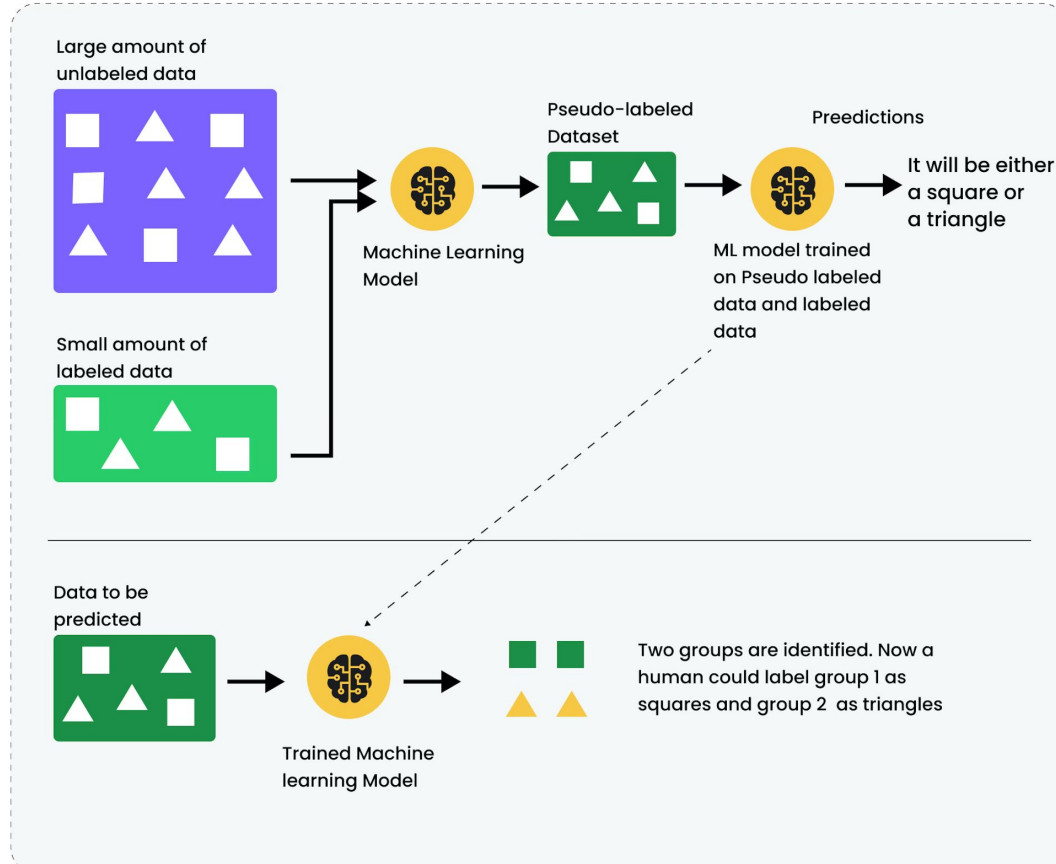
Reinforcement Learning



Semi-Supervised Learning

- ❖ Uses a small amount of labeled data combined with a large amount of unlabeled data during training.
- ❖ Why? Labeling data is expensive (requires human experts), but raw data is cheap.
- ❖ **Applications:**
 - Speech Analysis
 - Web Content Classification
 - Medical Images
- ❖ **Approaches**
 - Pseudo-Labeling
 - Generative Adversarial Networks (GANs)

Semi-Supervised Learning



Self-Supervised Learning

- ❖ Self-Supervised Learning is a technique where the model generates its own supervisory signals (labels) from the input data itself.
- ❖ It solves a "pretext task" (a made-up problem) to learn the underlying structure of the data, creating useful representations for downstream tasks.
- ❖ SSL is a middle ground between supervised learning (with labels) and unsupervised learning (without labels).
- ❖ **Applications:**
 - Natural Language Processing (NLP): Language Understanding, Text Generation
 - Computer Vision: Image Colorization, Medical Imaging
 - Robotics

Self-Supervised Learning

Pretrain



Representation



Pretext Task



Finetune



Representation

Classifier



Dog

Machine Learning Pipeline

- ❖ A Machine Learning Pipeline is an automated workflow that sequences all the steps required to turn raw data into a working prediction model.
- ❖ The machine learning pipeline consists of the following stages:
 - **Data preprocessing**
 - This is the foundational stage where raw, messy data is transformed into a clean, usable format.
 - Real-world data often contains errors, duplicates, or missing values, this stage involves data cleaning (correcting typos), imputation (filling in missing numbers with averages), and handling outliers that might skew results.
 - It also includes Exploratory Data Analysis (**EDA**) to understand data distribution and feature scaling to ensure all numerical variables (like age and salary) are on the same mathematical scale so the model can process them effectively.

Machine Learning Pipeline

❖ Training Models

- In this stage, the cleaned data is fed into a selected machine learning algorithm (such as Linear Regression or a Decision Tree) to learn patterns.
- The dataset is typically split into a training set (used for learning) and a validation set (used for checking progress).
- The algorithm iterates through the data, adjusting its internal parameters to minimize the loss function.
- The goal here is to create a baseline model that effectively captures the relationship between the input features and the target output.

Machine Learning Pipeline

❖ **Model Optimization**

- This stage focuses on refining it to achieve peak performance.
- This primarily involves hyperparameter tuning, where external settings of the algorithm are adjusted systematically using techniques like Grid Search.
- It may also involve feature selection, where weak or redundant features are removed to reduce noise.

❖ **Model Evaluation**

- The model is tested on a completely new test dataset that it has never seen before.
- Instead of just checking if the model works, we measure how well it works using specific metrics.
- For classification, we use a Confusion Matrix to calculate Accuracy, Precision, Recall, and F1-Score; for regression, we look at error rates like Mean Squared Error.

Machine Learning Pipeline

❖ **Model Deployment**

- The final stage involves taking the trained and evaluated model out of the development environment and integrating it into a production system where end-users can interact with it.
- This usually means wrapping the model in an API so that software applications can send data to it and receive predictions in real-time.
- Post-deployment, it is essential to set up monitoring systems to track the model's performance and detect "concept drift" (when real-world data changes), which signals when the model needs to be retrained.

Machine Learning Pipeline

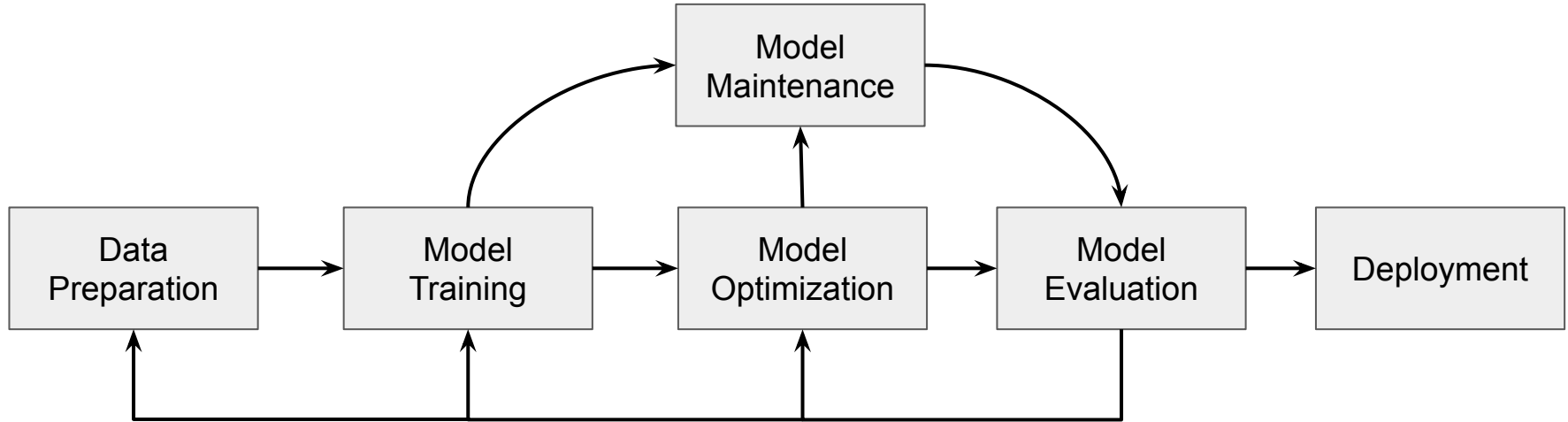


Fig: Machine Learning Pipeline

Four pillars of Machine Learning

❖ Linear Algebra:

- The branch of mathematics dealing with vector spaces and matrices. It provides the language and structure to represent, store, and manipulate massive datasets simultaneously.
- **Machine Learning Example:**
 - Image Processing & Neural Network Inputs.
 - An AI sees a black-and-white image not as a picture, but as a matrix (a grid) of pixel numbers.
 - When the image passes through a neural network layer, the model performs matrix multiplication—multiplying the input matrix by a "weight matrix"—to instantly detect features like edges, lines, and shapes.

Four pillars of Machine Learning

❖ Probability and Statistics

- The mathematical fields focused on analyzing data distributions, managing randomness, and quantifying uncertainty to make predictions from incomplete data.
- **Machine Learning Example:**
 - **Classification Confidence Scores.**
 - When a medical AI looks at an X-ray, it rarely says a definitive "**yes**" or "**no**."
 - Instead, it uses probability to output a confidence score (e.g., "There is an 88% probability this scan shows pneumonia").
 - This helps developers and doctors understand how reliable the model's prediction is.

Four pillars of Machine Learning

❖ Calculus

- The study of continuous change, using derivatives to measure how a change in an input variable impacts the final output.
- **Machine Learning Example:**
 - Backpropagation (Error Tracking).
 - When an AI makes a wrong prediction, the system needs to know which specific internal setting (weight) caused the mistake.
 - Calculus uses the chain rule to calculate derivatives, which act like a compass showing exactly how much the model's total error will go up or down if a specific setting is tweaked.

Four pillars of Machine Learning

❖ Optimization

- The process of adjusting inputs and parameters to find the absolute best possible result—usually by minimizing a loss function (error) or maximizing accuracy.
- **Machine Learning Example:**
 - **Gradient Descent (The Learning Process).**
 - Calculus tells the AI which direction the error changes, but Optimization is the actual engine that moves the model there.
 - An optimization algorithm like Gradient Descent takes the calculus data and systematically tweaks the model's weights step-by-step until the overall error rate is as close to zero as possible.

Linear Algebra

Introduction to Linear Algebra

- The branch of mathematics that studies **vector spaces** and **linear transformations** between vector spaces, such as rotating a shape, scaling it up or down, translating it (i.e. moving it), etc.
 - Vector spaces: A group of object called vector, added collectively and multiplied by numbers, called scalars.
 - Linear transformation: a rule that changes one vector into another while keeping the structure of the space intact. Like (rotating, scaling, reflection, shear, and projections)
- Machine Learning relies heavily on Linear Algebra

Scalar and Vector

- **Scalar:** A single numerical value, often representing magnitude or quality
- **Vector:** An ordered array of numbers, which can represent both magnitude and direction in space.

Why we need vector?

- Vectors are used to represent your information in a compact form.

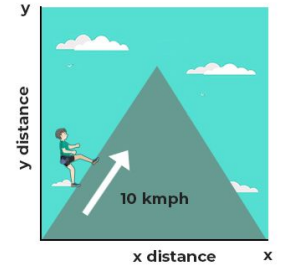
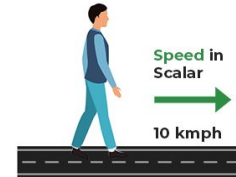
Example: I have a house with 4 rooms, 2 toilets, 10.2m²

$$house = \begin{pmatrix} 4 \\ 2 \\ 10.2 \end{pmatrix}$$

This is a three dimensional vectors because

It has three dimensions/features/explanatory variables/predictors (there are many names indeed) which are rooms, toilets, and size.

Scalar and Vector

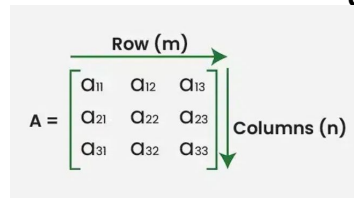


Velocity is vector which means direction is also included

Matrices

- A set of numbers arranged in rows and columns so as to form a rectangular array.

When we use Matrices?



A diagram showing a 3x3 matrix A enclosed in square brackets. The elements are labeled a_{11}, a_{12}, a_{13} in the first row, a_{21}, a_{22}, a_{23} in the second row, and a_{31}, a_{32}, a_{33} in the third row. A green arrow above the matrix points to the right and is labeled "Row (m)". A green arrow to the right of the matrix points downwards and is labeled "Columns (n)".

If we have more house, it can be represented through matrices like:

We have second house with room 2, 1 toilet, and size 4 m²

$$X = \begin{pmatrix} 4 & 2 \\ 2 & 1 \\ 10.2 & 4 \end{pmatrix}$$

- Things to consider:
 - Each sample/record/observations is represented as columns, while the dimensions/features are represented as a rows.
 - Its uses UPPERCASE. Example **X** or (**big X**)
 - **N** here is **2**, and N is often used to mean “number of samples/observations/record”

Normally, X has a shape of $\mathbf{m}$ features x $\mathbf{n}$ samples with the following structure:

The number located in the **ith** row, and **jth** column of a matrix is sometimes noted **$X_{\{i,j\}}$** or **X_{ij}**

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & x_{2,3} & \cdots & x_{2,n} \\ x_{3,1} & x_{3,2} & x_{3,3} & \cdots & x_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & x_{m,3} & \cdots & x_{m,n} \end{bmatrix}$$

Operations with matrices/vectors

- Scalar multiplication:

Multiply **X** with number 3: $3X = \begin{bmatrix} 3 \times 4 & 3 \times 2 \\ 3 \times 2 & 3 \times 1 \\ 3 \times 10.2 & 3 \times 4 \end{bmatrix}$

“3” is a scalar and often represented as lowercase with symbol like lambda, alpha, and gamma like this:

$$\alpha = 3 \text{ or } \lambda = 3 \text{ or } \gamma = 3$$

Dot product

- It is the product of the two vectors' Euclidean magnitudes and the cosine of the angle between them.
 - What if I want to multiply rooms by 2, toilets by 0.5, and size by 1 and add them together? You can do a dot product with a vector like this:

$$X^T \beta = \begin{bmatrix} 4 & 2 & 10.2 \\ 2 & 1 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 0.5 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \times 2 + 2 \times 0.5 + 10.2 \times 1 \\ 2 \times 2 + 1 \times 0.5 + 4 \times 1 \end{bmatrix}$$

- X is transpose first. Its is transpose to dot with β as two matrices multiplied only if first metrics column is same as second metrics row. Example X transpose has (2, 3) and β has (3,)
- Formula:

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \times \|\mathbf{v}\| \times \cos(\theta) \quad \text{Or} \quad \mathbf{u} \cdot \mathbf{v} = \sum_i \mathbf{u}_i \times \mathbf{v}_i$$

Main properties of dot product:

- Commutative: $u \cdot v = v \cdot u$
- Dot product is defined between only two vector not between scalar and vector
 - Example: $u \cdot v \cdot w$ where $u \cdot v$ is a scalar and w is a vector
- Not associative: $(u \cdot v) \cdot w \neq u \cdot (v \cdot w)$
- the dot product is associative with regards to scalar multiplication:
 - $\lambda \times (u \cdot v) = (\lambda \times u) \cdot v = u \cdot (\lambda \times v)$

For Matrix multiplication:

- Not commutative: $QR \neq RQ$
- Is associative: $Q(RS) = (QR)S$

Norm

- It is used to measure the length of the vectors (a.k.a the magnitude) of $\mathbf{u}$ vector.
- Norm of the vector is denoted by the $\| \mathbf{u} \|$.
- The Euclidean norm is defined as:

$$\| \mathbf{u} \| = \sqrt{\sum_i \mathbf{u}_i^2}$$

- In python:

$$\sqrt{x} = x^{\frac{1}{2}}$$

Type of matrices

1. Square matrix $\begin{bmatrix} 4 & 9 & 2 \\ 3 & 5 & 7 \\ 8 & 1 & 6 \end{bmatrix}$

2. Upper triangular matrix $\begin{bmatrix} 4 & 9 & 2 \\ 0 & 5 & 7 \\ 0 & 0 & 6 \end{bmatrix}$

3. Lower triangular matrix $\begin{bmatrix} 4 & 0 & 0 \\ 3 & 5 & 0 \\ 8 & 1 & 6 \end{bmatrix}$

4. Diagonal matrix $\begin{bmatrix} 4 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 6 \end{bmatrix}$

Inverse Metrics

- The inverse of a Matrix is the matrix that on multiplying with the original matrix results in an identity matrix.
- For any square matrix A, its inverse is denoted as A^{-1}

$$A.A^{-1} = A^{-1}A = I$$

Where, I is Identity matrix

$$A^{-1} = [1 / (ad-bc)] \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

- Singular Matrix
 - A matrix whose value of the determinant is zero is called a singular matrix. Inverse of singular matrix does not exist

Determinant

- A square matrix M , noted $\det(M)$ or $|M|$ is a value calculated from element $(M_{i,j})$

$$|M| = M_{1,1} \times |M^{(1,1)}| - M_{2,1} \times |M^{(2,1)}| + M_{3,1} \times |M^{(3,1)}| - M_{4,1} \times |M^{(4,1)}| + \dots \pm M_{n,1} \times |M^{(n,1)}|$$

Example:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

$$|M| = 1 \times \begin{vmatrix} 5 & 6 \\ 8 & 0 \end{vmatrix} - 2 \times \begin{vmatrix} 4 & 6 \\ 7 & 0 \end{vmatrix} + 3 \times \begin{vmatrix} 4 & 5 \\ 7 & 8 \end{vmatrix}$$

$$|M| = 1 \times (-48) - 2 \times (-42) + 3 \times (-3) = 27$$

Eigenvector and Eigenvalues

- An Eigenvector is a vector that maintains its direction after undergoing a linear transformation
- An Eigenvalue is the scalar value that the eigenvector was multiplied by during the linear transformation

Formular:

$$A \cdot v = \lambda \cdot v$$

Where A is matrix, v is eigenvector, and λ is eigenvalues

Linear Combination

- ❖ A linear combination is formed by taking a set of variables, multiplying each by a constant scale factor, and adding the results together.
- ❖ Mathematically, it must look exactly like this:

$$y = c_1 x_1 + c_2 x_2 + \dots + c_n x_n \dots\dots\dots (1)$$

➤ Where, C is constant and x is variables (input, vectors, metrics)

Continuation Optimization

- ❖ This is also called “**Homotopy Optimization**”.
- ❖ Continuous optimization is a branch of optimization in applied mathematics.
- ❖ A technique used to solve complex, non-convex mathematical problems.
- ❖ History:
 - **1950's, Davidenko differential equation;**
 - To solve complex systems of nonlinear equations
 - It wasn't about optimization at all.
 - **Computer vision, 1987:**
 - Optimization technique for **non-convex problems**
 - Smoothing a non-convex objective into an easy convex one and gradually morphing it back — was introduced by Blake and Zisserman in 1987 under the name "**homotopy optimization**," also called **continuation optimization**
 - It was first known as "**graduated non-convexity**," used for tasks like image restoration,

First Order Method Vs Second Order Method

First Order Method	Second Order Method
Gradient (∇f) — Vector of first partial derivatives.	Hessian (H) — Matrix of second partial derivatives.
Measures the slope and direction of steepest descent.	Measures the curvature (how fast the slope is changing).
Linear (Requires many iterations to get highly precise).	Quadratic (Doubles the digits of precision every step near a minimum).
Extremely sensitive to the learning rate.	Naturally calculates the optimal step size via curvature.
Per-iteration Complexity: $O(d)$ where d is the number of parameters (Very cheap).	Per-iteration Complexity: $O(d^3)$ to compute and invert the Hessian matrix (Catastrophically expensive).

Why Do We Need Derivatives in Deep Learning?

To minimize the loss, we need to know

- ❖ Which direction should the parameters move?
- ❖ How much should they move?
- ❖ Is the surface flat or steep?
- ❖ Are we at a minimum or merely on a plateau?

These questions are answered by **derivatives**.

Unconstrained Optimization

- ❖ The mathematical problem of finding the point at which a real-valued **objective function** reaches its lowest (or highest) possible value, where the decision variables are completely free to take on any real value without any restrictions, boundaries, or external conditions.
- ❖ Mathematically, given an objective function $f: \mathbf{R}^n \rightarrow \mathbf{R}$, the unconstrained minimization problem is formulated as:
- ❖ Where, x is variable vector
- ❖ $f(x)$ objective function

$$\min_{x \in \mathbb{R}^n} f(x)$$

(**Note:** Maximization problems $\max f(x)$ are mathematically equivalent, as they can be solved by minimizing the negative of the function: $\min -f(x)$).

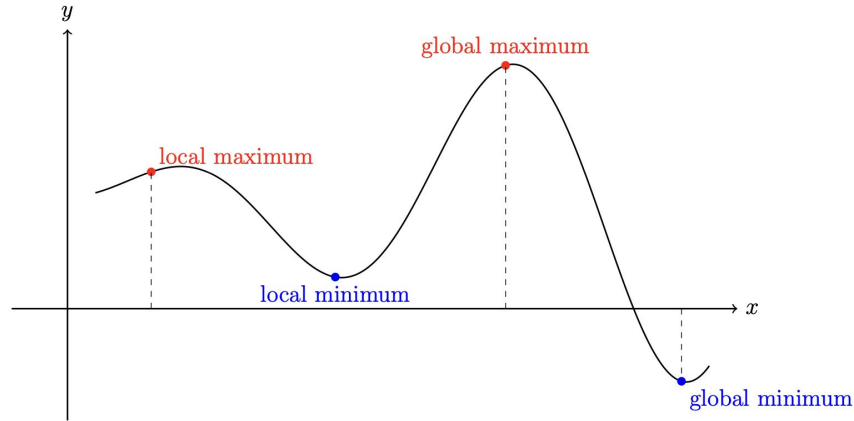
Types of Solution Optimization

❖ Global Minimum

- A point is a global minimum if it yields the absolute lowest value of the function over the entire space:

❖ Local Minimum

- A point is a local minimum if it yields the lowest value within its immediate neighborhood



Gradient Descent

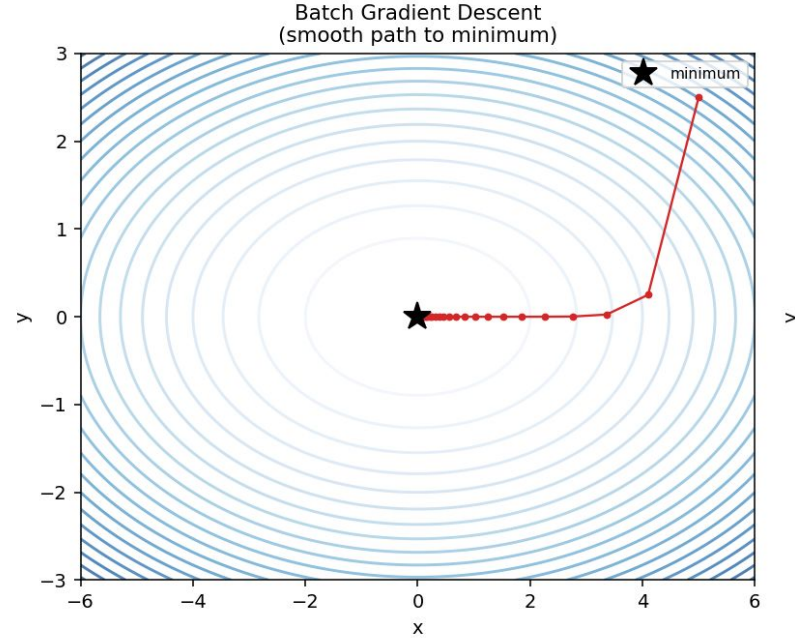
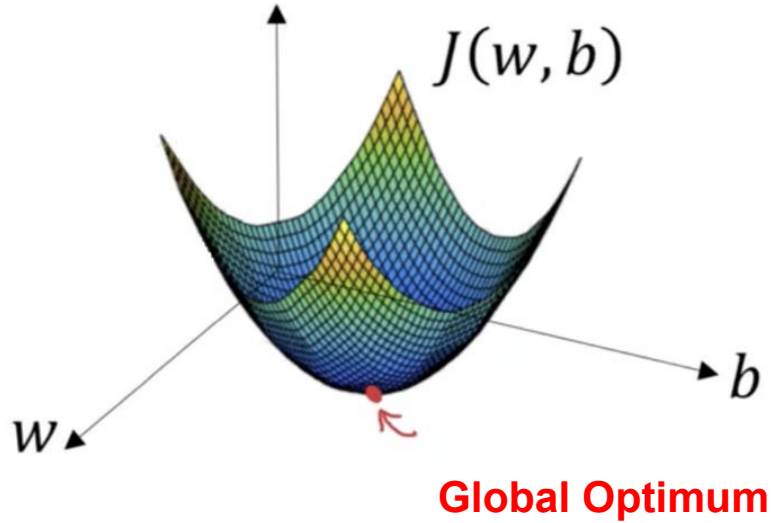
- ❖ The negative gradient points toward steepest decrease — so stepping repeatedly in that direction walks downhill to a minimum.

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \eta \nabla f(\mathbf{x}_i)$$

η = learning rate (step size)

- ❖ Minimizes the loss or error of a model
- ❖ Updates parameters step by step using gradients
- ❖ Helps models like neural networks and regression learn optimal weights
- ❖ Variants include **Batch Gradient Descent**, **Stochastic Gradient Descent** and **Mini Batch Gradient Descent**

Gradient Descent



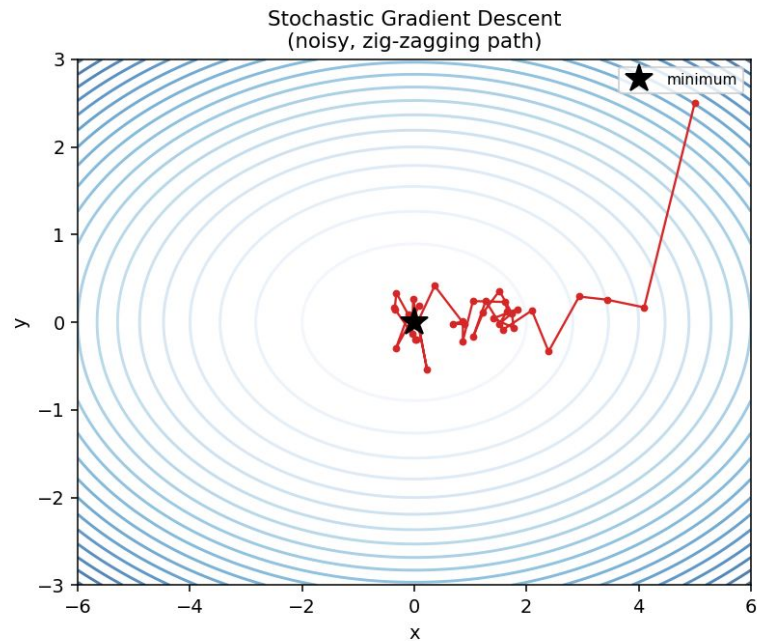
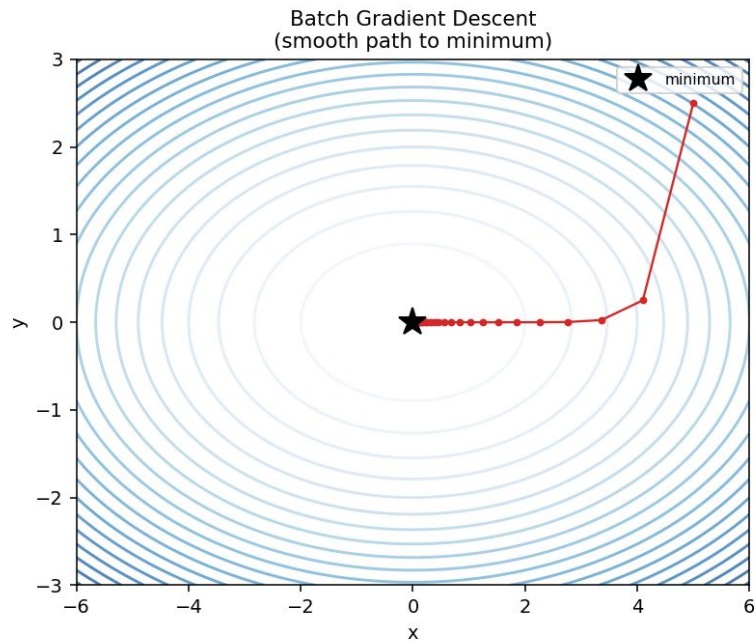
Stochastic Gradient Descent (SGD)

- ❖ **Stochastic Gradient Descent** is an optimization algorithm used in machine learning, especially for large datasets, that updates model parameters efficiently using small batches or single samples.

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \eta \nabla f_i(\mathbf{x}_i)$$

- ❖ Variant of gradient descent designed for faster and scalable learning
- ❖ Updates parameters using one data point or small batches at a time
- ❖ Reduces computation compared to full batch gradient descent
- ❖ Widely used in deep learning for efficient training

Stochastic Gradient Descent (SGD)



Beyond Gradient Descent

- ❖ Optimization methods that go beyond standard first-order gradient descent
- ❖ Additional information (such as curvature, momentum, or adaptive learning rates) or by employing entirely different optimization strategies.
- ❖ These methods are often faster, more stable, or better at escaping poor local solutions.

Why go beyond Gradient Descent?

- Gradient descent has several limitations:
- Can converge slowly.
- Sensitive to learning rate.
- Can get stuck in local minima or saddle points.
- Zig-zagging in narrow valleys.
- Does not use curvature information.

The Jacobian: derivatives of vector-valued functions

- ❖ The gradient is insufficient because it only describes the derivative of a single output.
- ❖ The Jacobian matrix extends the concept of derivatives to vector-valued functions.
- ❖ the Jacobian acts as the best linear approximation of a nonlinear function near the current point. i.e. $\nabla y = J \nabla x$
- ❖ In Backpropagation, Each derivative between vectors is a Jacobian matrix.

Why Is Jacobian Important?

- which input influences which output,
- the strength of that influence,
- and the direction of the change.

$$\mathbf{J}_f(x, y) = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} :$$

Hessian Matrix

- ❖ The gradient is responsible for, "Which direction should we move?"
- ❖ It does not tell us
 - how sharply the function bends,
 - whether we are at a minimum, maximum, or saddle point, or how quickly the slope changes.
- ❖ The Hessian matrix answers these questions.
- ❖ The Hessian matrix is the matrix of all second-order partial derivatives of a scalar-valued function.
- ❖ The Hessian provides information about:
 - the curvature of the loss landscape,
 - parameter interactions,
 - whether the optimizer is near a minimum or saddle point,
 - and the confidence or uncertainty of parameter estimates (e.g., in logistic regression).
- ❖ Limitation:
 - Requires computing second-order derivatives.
 - Memory requirement grows as n^2 , where n is the number of parameters.
 - Matrix inversion is computationally expensive ($O(n^3)$).
 - Impractical for very large deep neural networks, so approximations are typically used.

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Constraint Optimization

- ❖ Constraint optimization is the process of finding the optimal value (minimum or maximum) of an objective function while satisfying one or more constraints.
- ❖ Mathematical form,

$$\min f(\mathbf{x}) \quad \text{s.t.} \quad g_i(\mathbf{x}) = 0, \quad h_j(\mathbf{x}) \leq 0$$

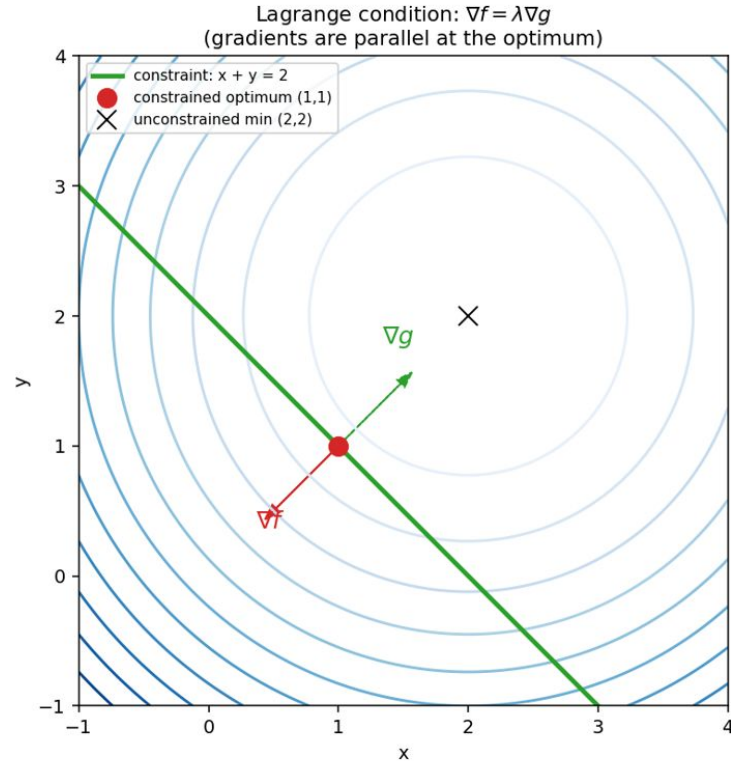
Why Do We Need Constraints?

- ❖ Overfitting
- ❖ Very large weights
- ❖ High memory consumption
- ❖ Slow inference
- ❖ Unstable training
- ❖ Poor generalization

Constraints help produce models that are not only accurate but also efficient, robust, and practical.

Constraint Optimization

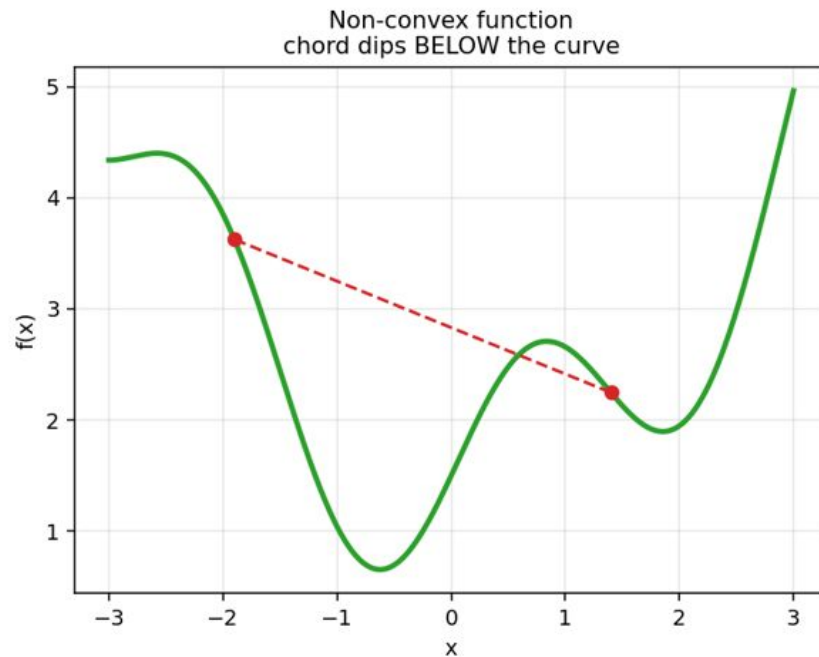
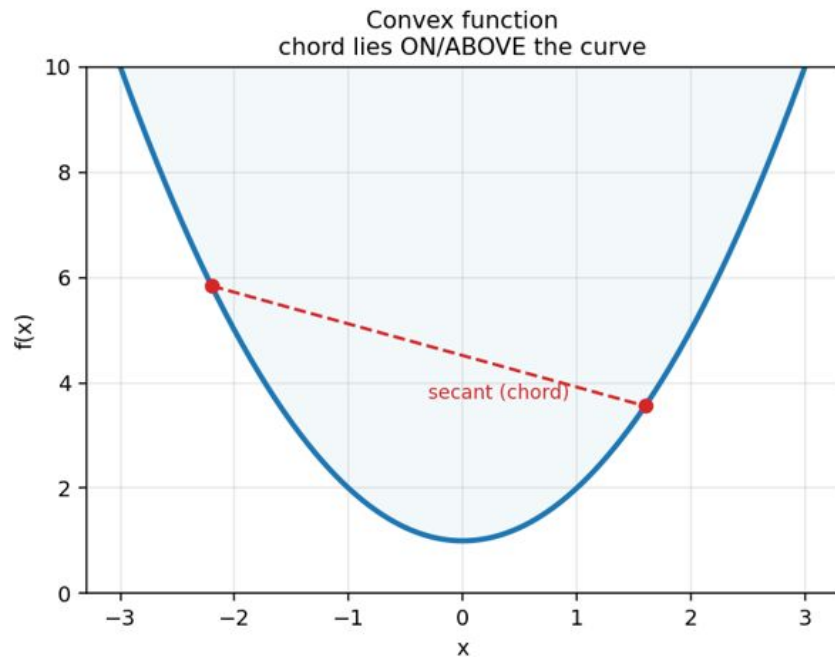
Lagrange condition



Convex Optimization

- ❖ Convex optimization is one of the most important areas of optimization because many machine learning algorithms are formulated as convex optimization problems.
- ❖ A function is said to be convex if the line segment between any two points on its graph lies above or on the graph itself.
- ❖ Its main advantage is that every local optimum is also a global optimum, making optimization more reliable and mathematically tractable.
- ❖ **Convex Sets:**
 - A convex set is a subset of a vector space that has a specific property: for any two points within the set, the line segment connecting them lies entirely within the set.
 - A set C in a vector space is called convex if, for any two points $x, y \in C$, the line segment joining them is also in C . Formally,
 - $C = \{\lambda x + (1-\lambda)y \mid 0 \leq \lambda \leq 1, x, y \in C\}$

Convex Optimization



Discriminative Models

❖ Discriminative Models

- Discriminative models learn the decision boundary between classes.
- They focus on modeling the conditional probability $P(y|x)$ —essentially asking, "Given this data x , what is the probability it belongs to class y ?"
 - **Goal:** To discriminate (distinguish) between different classes.
 - **How it works:** It ignores how the data was generated and focuses strictly on what separates one group from another.
 - **Analogy:** If you want to distinguish between Spanish and French, a discriminative model learns the specific linguistic patterns that make them different.
 - **Examples:**
 - Logistic Regression
 - Support Vector Machines (SVM)
 - Neural Networks
 - Decision Trees

Generative Models

- ❖ Generative models learn the distribution of the individual classes.
- ❖ They model the joint probability $P(x, y)$, which can be transformed using Bayes' Theorem to find $P(y|x)$.
- ❖ They essentially ask, "What does class A look like, and what does class B look like?"
 - **Goal:** To understand the underlying distribution of the data. Because they "understand" the data, they can often generate new, synthetic data points that look like the original set.
 - **How it works:** It learns the characteristics of each class.
 - **Analogy:** To distinguish Spanish from French, a generative model would actually learn how to speak Spanish and how to speak French.
 - **Examples:**
 - Naive Bayes
 - Gaussian Mixture Models (GMM)
 - Hidden Markov Models (HMM)
 - Generative Adversarial Networks (GANs)

Learning vs. Generalizing

- ❖ When you train a model, you are adjusting its internal parameters to minimize error.
- ❖ However, the goal isn't just to perform well on the data it has already seen, but to generalize to new, unseen data.
 - Training: The model looks at both features and labels to find patterns.
 - Validation: You hide the labels from the model and ask it to predict them. This provides an unbiased look at how the model is doing during the development phase.

The Data Split (Train, Validation, Test)

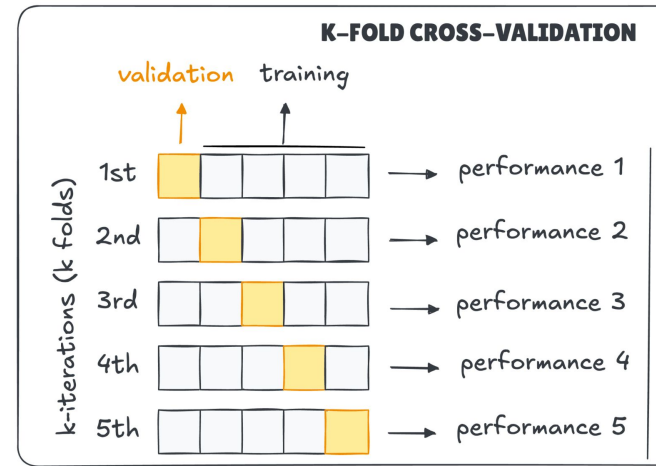
- ❖ To evaluate a model fairly, you cannot test it on the same data it used for learning. We typically split data into three distinct buckets:
 - Training Set (60-70%): The "Textbook." The model looks at these features and labels to adjust its internal weights.
 - Validation Set (15-20%): The "Practice Quiz." You use this during the development phase to compare different models or tune settings (Hyperparameters).
 - Test Set (15-20%): The "Final Exam." This data is kept in a "vault" and only used once the model is finished to see how it performs in the real world.

Hyperparameter Tuning

- ❖ In Machine Learning, there are two types of parameters:
 - **Model Parameters:** Learned automatically by the model during training (e.g., weights in a neural network).
 - **Hyperparameters:** Settings you choose before training starts (e.g., how deep a tree should grow, or how fast the model should learn).
- ❖ Common Tuning Strategies:
 - **Grid Search:** You provide a list of values for each hyperparameter, and the computer tries every single combination. It's thorough but slow.
 - **Random Search:** The computer picks random combinations from a range. Surprisingly, this is often faster and just as effective as Grid Search.
 - **Bayesian Optimization:** A "smart" search that uses past results to pick the next best hyperparameter to try.

Cross-Validation (K-Fold)

- ❖ What if your random split put all the "easy" data in the training set and the "hard" data in the test set? Your results would be biased. K-Fold Cross-Validation solves this.
 - Split the data into K equal parts (folds).
 - Train on K-1 folds and validate on the remaining 1 fold.
 - Repeat this K times, so every data point gets a chance to be in the validation set.
 - Average the scores.
- ❖ **Early Stopping**
 - This is a technique used primarily in Neural Networks.
 - You monitor the validation loss during training.
 - If the validation loss stops improving (or starts increasing) for a set number of rounds, you stop the training automatically.
 - This prevents overfitting.



Training and Validation Loss

- ❖ Loss is a numerical measure of how far off a model's prediction is from the actual truth.
- ❖ Think of it as a "penalty score": the higher the loss, the worse the model is performing.
- ❖ The goal of training a machine learning model is to minimize this loss through a process called Optimization.
- ❖ **Common Loss Functions:** Mean Squared Error (MSE), Mean Absolute Error (MAE), Binary Cross-Entropy (Log Loss), Categorical Cross-Entropy
 - **Training Loss:** This measures the error of the model on the data it is currently learning from. It is the "feedback" the model uses to update its weights during the training process.
 - **Validation Loss:** This measures the error on a separate set of data that the model has never seen during training. It represents how well the model can generalize to new, unseen information.

Bias and Variance

❖ Bias (The "Assumptions" Error)

- Bias is the error introduced by approximating a real-world problem (which is usually complex) with a much simpler model.
- It represents how far off the average prediction is from the actual value.
- **Higher Bias:** The model makes heavy assumptions. It is too "rigid." Even with more data, it refuses to learn the complexities.
 - Result: Underfitting. The model is consistently wrong on both training and test data.
 - Example: Using a straight line (Linear Regression) to predict a curvy relationship.
- **Lower Bias:** The model makes fewer assumptions about the data. It is flexible and can capture complex relationships.
 - Result: Potentially a Good Fit, but if too low, it can lead to overfitting.

Bias and Variance

❖ Variance (The "Sensitivity" Error)

- Variance is the error introduced by the model's sensitivity to small fluctuations in the training set.
- It represents how much the model would change if you used a different training set.
- **Higher Variance:** The model is "too flexible." It pays too much attention to individual data points, including noise and outliers.
 - Result: Overfitting. The model performs perfectly on training data but fails miserably on new, unseen data.
 - Example: A Decision Tree that is grown so deep it has a leaf for every single data point.
- **Lower Variance:** The model is stable. If you change the training data slightly, the model's predictions don't change much.
 - Result: The model is more robust, but if too low, it might be too simple (High Bias).

Learning Curve

❖ Underfitting (High Bias)

- In this scenario, both the training loss and validation loss remain high.
- The model is too simple to capture the underlying patterns in the data.
 - **Visual:** Both curves flatten out at a high error rate.
 - **Diagnosis:** The model hasn't learned enough.
 - **Fix:** Increase model complexity or add more features.

❖ Overfitting (High Variance)

- The training loss continues to decrease, but the validation loss starts to increase after a certain point.
- The model has "memorized" the noise in the training data rather than learning the general rules.
 - **Visual:** A widening gap appears between the two curves.
 - **Diagnosis:** The model is "cheating" by memorizing the training set.
 - **Fix:** Use regularization, dropout, or get more training data.

Learning Curve

❖ Optimal Fit (The Sweet Spot)

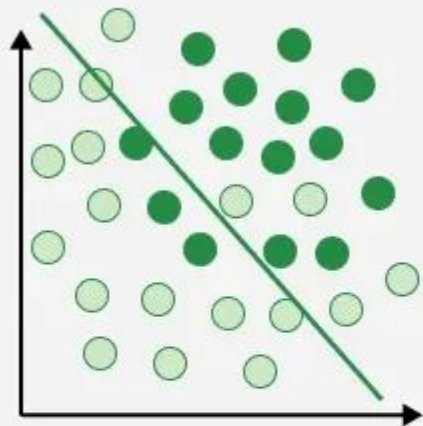
- Both training and validation loss decrease and eventually stabilize at a low level with a very small gap between them.
- Visual: The curves move downward together and plateau near each other.

❖ The Bias-Variance Tradeoff

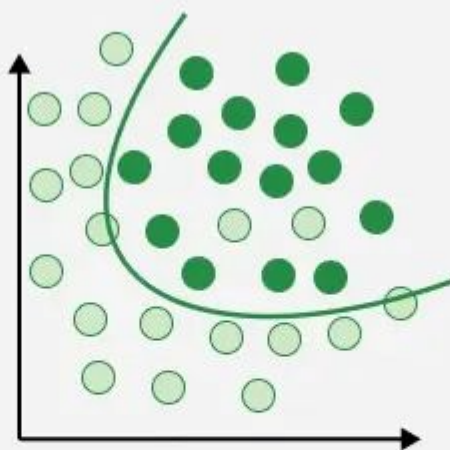
- The Bias-Variance Tradeoff is the "tug-of-war" in machine learning.

$$\text{Total Error} = (\text{Bias})^2 + \text{Variance} + \text{Irreducible Error}$$

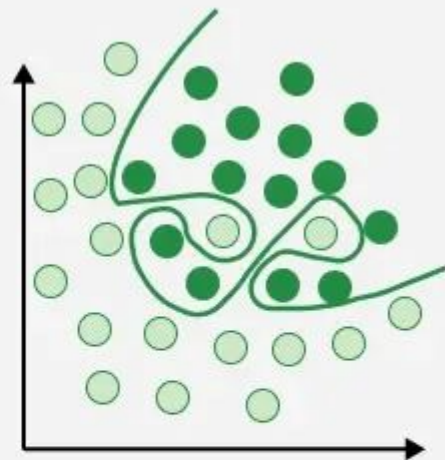
- Increasing model complexity reduces bias but increases variance (risk of overfitting).
- Simplifying the model reduces variance but increases bias (risk of underfitting).
 - **Low Bias / High Variance:** The model is highly flexible but inconsistent.
 - **High Bias / Low Variance:** The model is consistent but consistently wrong.
 - **The Sweet Spot:** The point where the total error is at its minimum.



Under-fitting



Appropriate-fitting

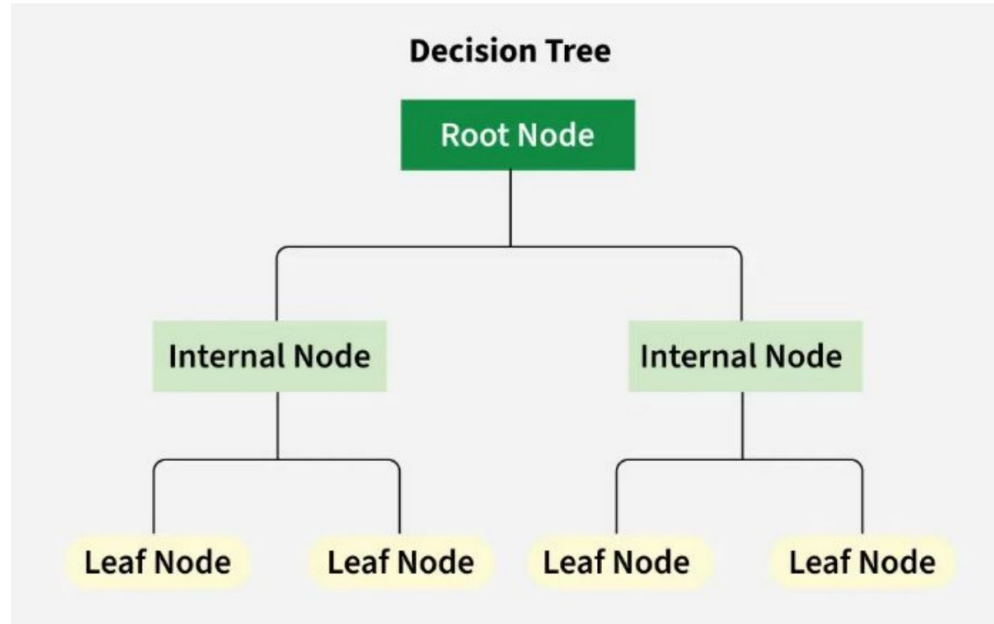


Over-fitting

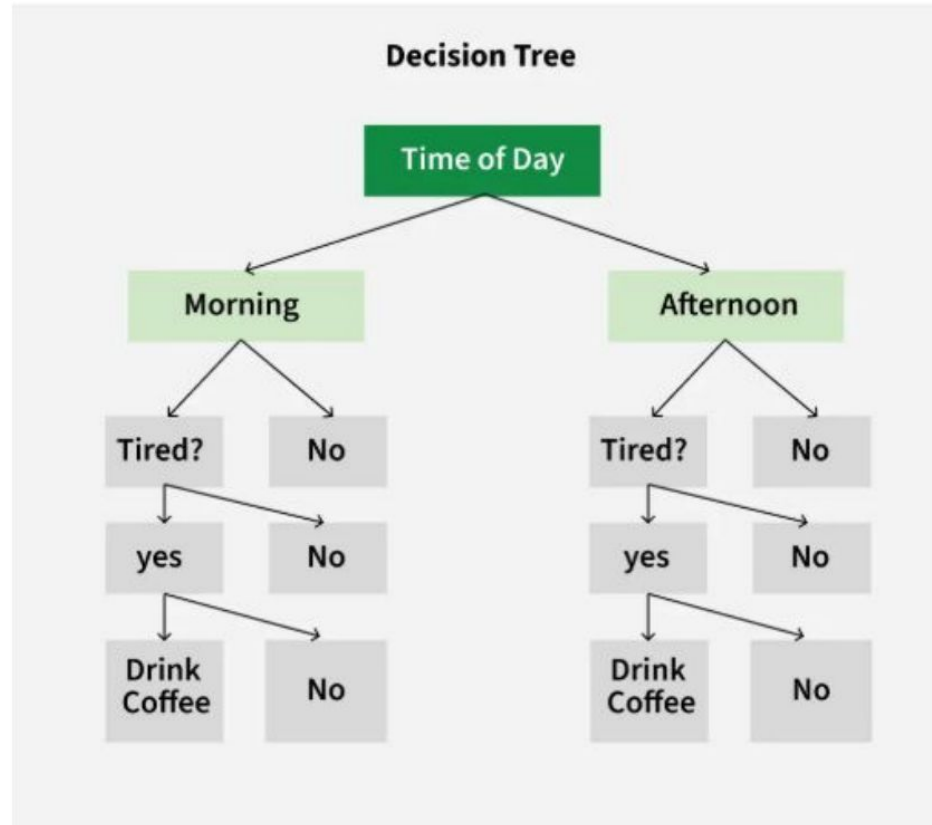
Decision Tree

- ❖ A Decision Tree helps us to make decisions by mapping out different choices and their possible outcomes.
- ❖ It has a tree-like structure that starts with one main question called the root node which represents the entire dataset.
 - Root Node: Starting point representing the whole dataset.
 - Branches: Lines connecting nodes showing the flow from one decision to another.
 - Internal Nodes: Points where decisions are made based on data features.
 - Leaf Nodes: End points of the tree where the final decision or prediction is made.
- ❖ Types of Decision Trees:
 - **Classification Trees:** Used for predicting categorical outcomes like spam or not spam. These trees split the data based on features to classify data into predefined categories.
 - **Regression Trees:** Used for predicting continuous outcomes like predicting house prices. Instead of assigning categories, it provides numerical predictions based on the input features.

Decision Tree:



Structure of a Decision Tree



Key Concepts

- ❖ **Splitting:** The process of dividing the dataset into subsets based on a feature value.
- ❖ **Impurity:** A measure of how mixed the classes are in a node. Common metrics to calculate impurity include:
 - Gini Impurity
 - Entropy (Information Gain)
 - Variance Reduction (for regression problems)
- ❖ **Pruning:** The process of removing parts of the tree that do not provide additional predictive power.

ID3 (Iterative Dichotomiser 3)

- ❖ The **ID3 algorithm** is a popular **decision tree algorithm** used in **machine learning**.
- ❖ It aims to **build a decision tree** by iteratively **selecting the best attribute** to **split the data** based on **information gain**.
- ❖ Each **node** represents a **test** on an **attribute**, and each branch represents a possible outcome of the test.
- ❖ It was developed in **1986** by **Ross Quinlan**.
- ❖ It employs a **top-down approach**, recursively selecting features to split the dataset based on **information gain**.

Numerical

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Numerical

Values (Outlook) = Sunny, Overcast, Rain

Step 1:

Calculate the Total Entropy of the Dataset

- ❖ First, we look at the target column: **Play Tennis**.
- ❖ Total days (m) = 14
- ❖ Yes = **9** days (D3, D4, D5, D7, D9, D10, D11, D12, D13)
- ❖ No = **5** days (D1, D2, D6, D8, D14)

$$H(S) = - \left(\frac{9}{14} \right) \log_2 \left(\frac{9}{14} \right) - \left(\frac{5}{14} \right) \log_2 \left(\frac{5}{14} \right) \approx \mathbf{0.940}$$

Numerical

Step 2: Calculate Information Gain for Each Feature

❖ Outlook Split:

- Overcast: **4 days** (D3, D7, D12, D13). All are Yes.
 - Entropy = 0 (It's perfectly pure).
- Sunny: **5 days** (D1, D2, D8, D9, D11). **2 are Yes, 3 are No.**
 - Entropy = $-(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$
- Rain: **5 days** (D4, D5, D6, D10, D14). **3 are Yes, 2 are No.**
 - Entropy = $-(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971$

❖ Information Gain (IG) = Entropy(S) - $\sum_{v \in (\text{Sunny, Overcast, Rain})} |S_v| / |S|$

$$IG(\text{Outlook}) = 0.940 - \left[\frac{4}{14}(0) + \frac{5}{14}(0.971) + \frac{5}{14}(0.971) \right] = \mathbf{0.246}$$

Numerical

Step 2: Calculate Information Gain for Each Feature

❖ Temp Split:

➤ Hot: **4 days** (). **2** are **Yes**, **2** are **No**.

■ Entropy = **1** (It's perfectly pure).

➤ Mild: **6 days** (). **4** are **Yes**, **2** are **No**.

■ Entropy = $-(4/6) \cdot \log_2(4/6) - (2/6) \cdot \log_2(2/6) = \mathbf{0.9183}$

➤ Cool: **4 days** (). **3** are **Yes**, **1** are **No**.

■ Entropy = $-(3/4) \cdot \log_2(3/4) - (1/4) \cdot \log_2(1/4) = \mathbf{0.8113}$

❖ **Information Gain (IG) = Entropy(S) - $\sum_{v \in (\text{Hot, Mild, Cool})} |S_v| / |S|$**

Or, Gain (S, Temp) = $0.94 - (4/14) \cdot 1 - (6/14) \cdot 0.9183 - (4/14) \cdot 0.8113 = \mathbf{0.0289}$

Numerical

Step 2: Calculate Information Gain for Each Feature

❖ Humidity Split:

➤ High: **7 days** (). **3** are **Yes**, **4** are **No**.

■ Entropy = $-(3/7) \cdot \log_2(3/7) - (4/7) \cdot \log_2(4/7) = \mathbf{0.9852}$

➤ Normal: **7 days** (). **6** are **Yes**, **1** are **No**.

■ Entropy = $-(6/7) \cdot \log_2(6/7) - (1/7) \cdot \log_2(1/7) = \mathbf{0.5916}$

❖ **Information Gain (IG) = Entropy(S) - $\sum_{v \in (\text{High, Normal})} |S_v| / |S|$**

Gain (S, humidity) = $0.94 - (7/14) \cdot 0.9852 - (7/14) \cdot 0.5916 = \mathbf{0.1516}$

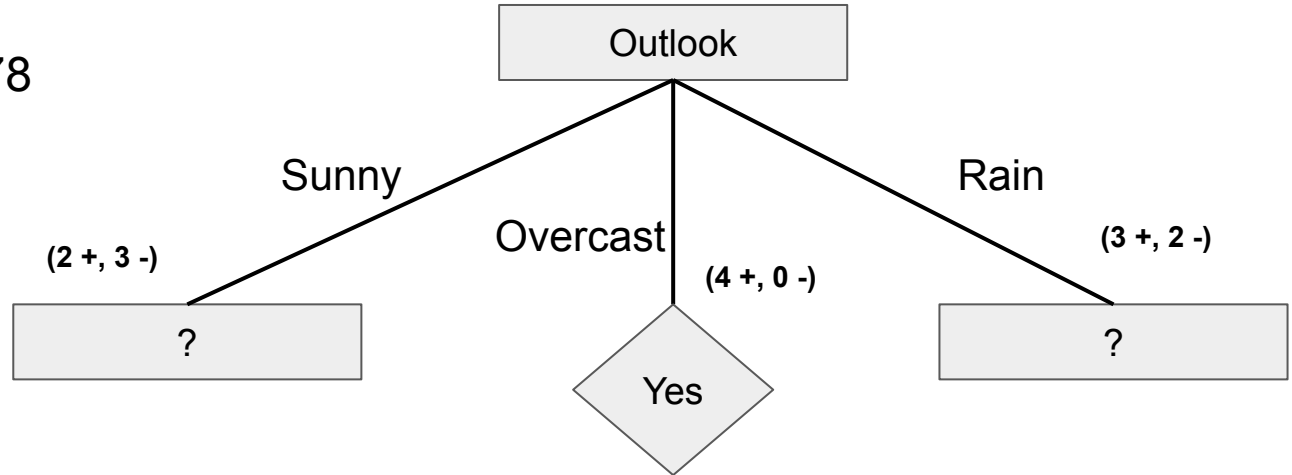
Numerical

Gain (S, Outlook) = **0.2464** (Higher means in the Root)

Gain (S, Temp) = 0.0289

Gain (S, Humidity) = 0.1516

Gain (S, Wind) = 0.0478



Numerical

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Numerical

$$\text{Entropy sunny } (S_{\text{sunny}}) = - \left(\frac{2}{5}\right) * \log_2 \left(\frac{2}{5}\right) - \left(\frac{3}{5}\right) * \log_2 \left(\frac{3}{5}\right) = \mathbf{0.97}$$

Value (Temp) = Hot, Mild, Cool

Split the data

❖ Hot: 0 yes and 2 No

➤ Entropy (S_{Hot}) = 0

❖ Mild: 1 yes and 1 No

➤ $S_{\text{mild}} = 1$

❖ Cool: 1 yes and 0 No

➤ $S_{\text{cool}} = 0$

$$\mathbf{\text{Gain}(S_{\text{sunny}}, \text{Temp}) = 0.97 - \frac{2}{5} * 0 - \frac{2}{5} * 1 - \frac{1}{5} * 0 = 0.570}$$

Numerical

Value (Humidity) = High, Normal

Split the data

❖ **High:** 0 yes and 3 No

➤ Entropy (S_{High}) = 0

❖ **Normal:** 2 yes and 0 No

➤ $S_{\text{Normal}} = 0$

❖ **Gain(S_{sunny} , Humidity) = $0.97 - \frac{3}{5} * 0 - \frac{2}{5} * 0 = 0.97$**

Numerical

Value (Wind) = Strong, Weak

Split the data

❖ **Strong:** 1 yes and 1 No

➤ Entropy (S_{strong}) = 1

❖ **Normal:** 1 yes and 2 No

➤ $S_{\text{Normal}} = 0.9183$

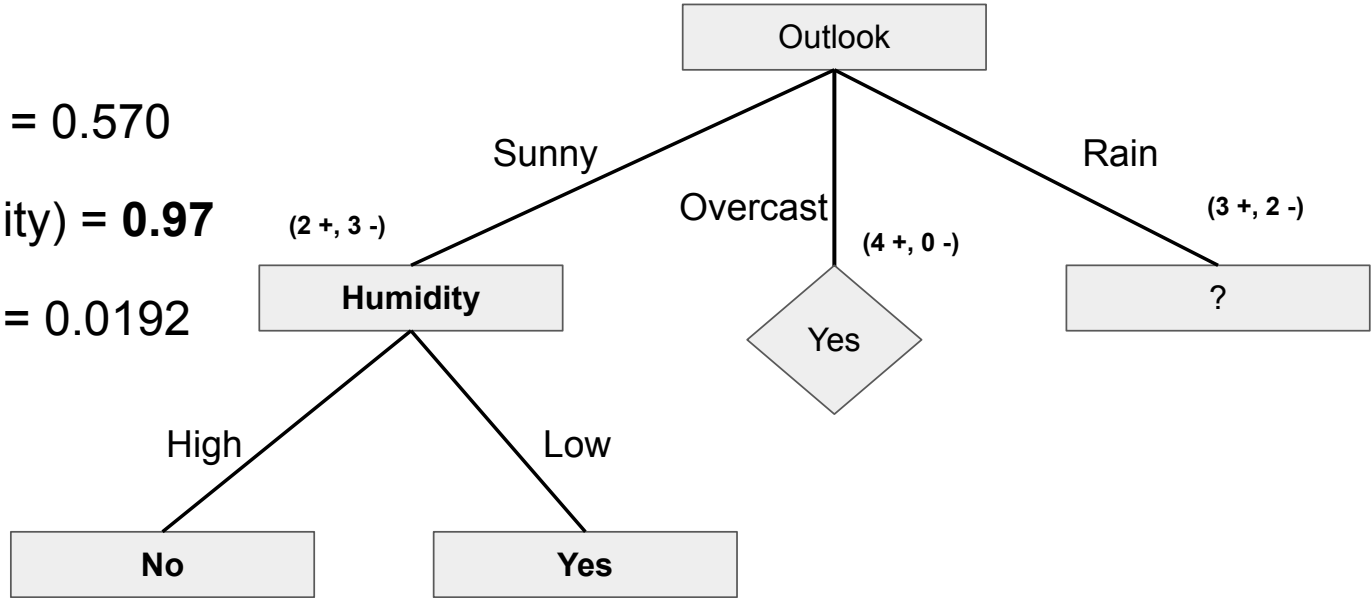
❖ **Gain(S_{sunny} , Wind) = $0.97 - \frac{2}{5} * 1.0 - \frac{3}{5} * 0.9183 = 0.0192$**

Numerical

Gain (S_{sunny} , Temp) = 0.570

Gain (S_{sunny} , Humidity) = **0.97**

Gain (S_{sunny} , Wind) = 0.0192



Numerical

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Numerical

$$\text{Entropy sunny } (S_{\text{sunny}}) = - \left(\frac{3}{5}\right) * \log_2 \left(\frac{3}{5}\right) - \left(\frac{2}{5}\right) * \log_2 \left(\frac{2}{5}\right) = \mathbf{0.97}$$

Value (Temp) = Hot, Mild, Cool

Split the data

- ❖ Hot: 0 yes and 0 No
 - Entropy (S_{Hot}) = 0
- ❖ Mild: 2 yes and 1 No
 - $S_{\text{mild}} = 0.9183$
- ❖ Cool: 1 yes and 1 No
 - $S_{\text{cool}} = 1$
- ❖ **Gain(S_{rain} , Temp) = $0.97 - \frac{0}{5} * 0 - \frac{3}{5} * 0.9183 - \frac{2}{5} * 1 = 0.0192$**

Numerical

Value (Humidity) = High, Normal

Split the data

❖ **High:** 1 yes and 1 No

➤ Entropy (S_{High}) = 1

❖ **Normal:** 2 yes and 1 No

➤ $S_{\text{Normal}} = 0.9183$

❖ **Gain(S_{rain} , Humidity) = $0.97 - \frac{2}{5} * 1 - \frac{3}{5} * 0.9183 = 0.0192$**

Numerical

Value (Wind) = Strong, Weak

Split the data

❖ **Strong:** 0 yes and 2 No

➤ Entropy (S_{strong}) = 0

❖ **Normal:** 3 yes and 0 No

➤ $S_{\text{Normal}} = 0$

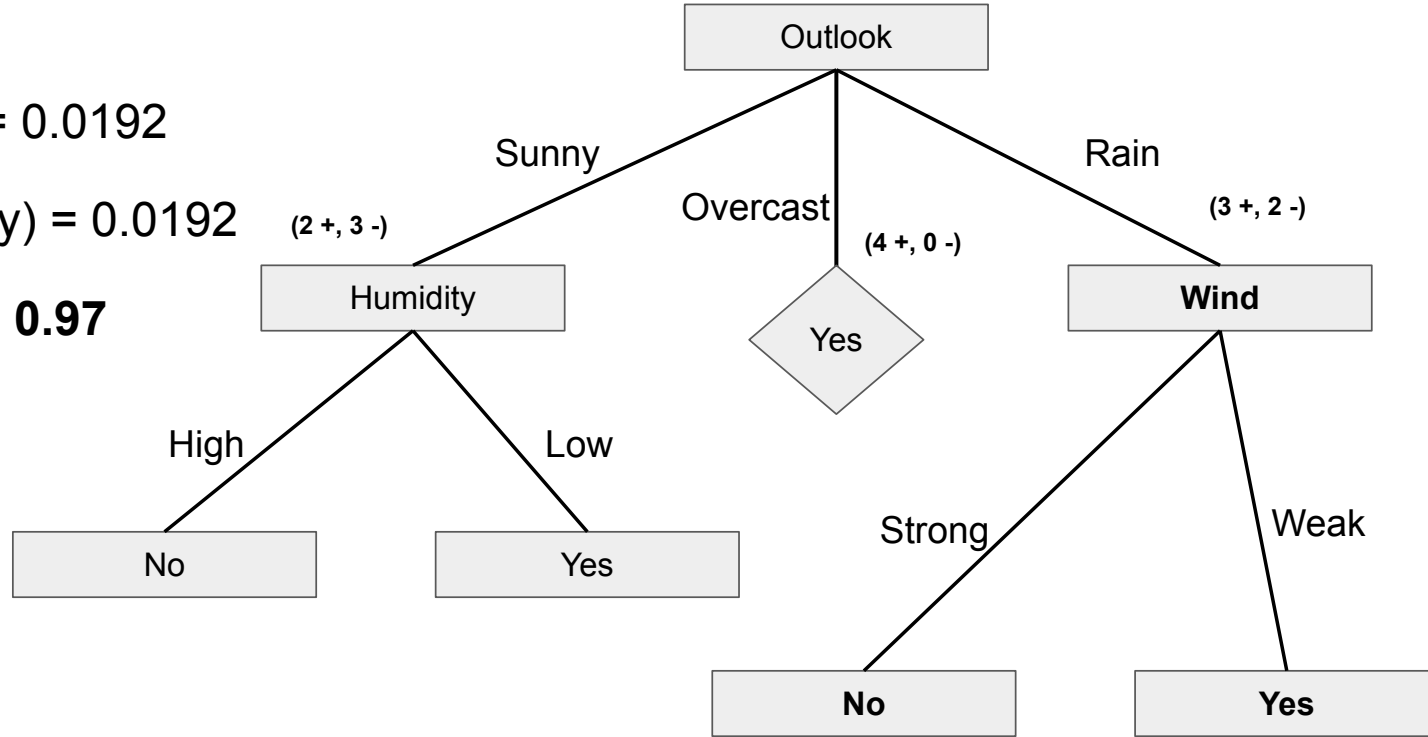
❖ **Gain(S_{rain} , Wind) = $0.97 - \frac{2}{5} * 0 - \frac{3}{5} * 0 = 0.97$**

Numerical

Gain (S_{rain} , Temp) = 0.0192

Gain (S_{rain} , Humidity) = 0.0192

Gain (S_{rain} , Wind) = **0.97**



Naive Bayes

- ❖ Naive Bayes is a classification algorithm based on Bayes' Theorem.
- ❖ It is called "naive" because it makes the massive simplification that all features are independent of each other.
- ❖ Naive Bayes performs well in many real-world applications such as spam filtering, document categorisation and sentiment analysis.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \text{ where } P(B) \neq 0$$

Where:

- ❖ A (Hypothesis): What we want to prove (e.g., "Patient has Cancer").
- ❖ B(Evidence): The new data we observed (e.g., "Positive Lab Test").
- ❖ P(A) (Prior): Initial belief probability.
- ❖ P(B|A) (Likelihood): The probability of seeing this evidence if the hypothesis is true.
- ❖ P(B) (Marginal Evidence): The total probability of the evidence occurring (normalizing constant).

Assumption of Naive Bayes

- ❖ **Feature independence:** This means that when we are trying to classify something, we assume that each feature (or piece of information) in the data does not affect any other feature.
- ❖ **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.
- ❖ **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- ❖ **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- ❖ **No missing data:** The data should not contain any missing values.

Numerical

RID	Age	Income	Student	Credit	Buy
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	Middle-aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle-aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle-aged	medium	no	excellent	yes
13	middle-aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Numerical

Predict class level of the tuple: $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$ using Bayesian classification.

Solution

Prior probability of each class can be computed based on the training tuples:

$$P(\text{buys} = \text{yes}) = 9/14 = 0.643$$

$$P(\text{buys} = \text{no}) = 5/14 = 0.357$$

Numerical

For Class C_1 (buys = yes):

- $P(\text{age} = \text{youth} | \text{buys} = \text{yes}) = 2/9 = 0.222$
- $P(\text{income} = \text{medium} | \text{buys} = \text{yes}) = 4/9 = 0.444$
- $P(\text{student} = \text{yes} | \text{buys} = \text{yes}) = 6/9 = 0.667$
- $P(\text{credit_rating} = \text{fair} | \text{buys} = \text{yes}) = 6/9 = 0.667$

For Class C_2 (buys = no):

- $P(\text{age} = \text{youth} | \text{buys} = \text{no}) = 3/5 = 0.6$
- $P(\text{income} = \text{medium} | \text{buys} = \text{no}) = 2/5 = 0.4$
- $P(\text{student} = \text{yes} | \text{buys} = \text{no}) = 1/5 = 0.2$
- $P(\text{credit_rating} = \text{fair} | \text{buys} = \text{no}) = 2/5 = 0.4$

Numerical

$P(X|\text{buys}=\text{yes})$

$$\begin{aligned} &= P(\text{age}=\text{youth}|\text{buys}=\text{yes}) * P(\text{income}=\text{medium}|\text{buys} = \text{yes}) * \\ &P(\text{student}=\text{yes}|\text{buys}=\text{yes}) * P(\text{credit_rating}=\text{fair}/\text{buys}=\text{yes}) \\ &= 0.222 * 0.444 * 0.667 * 0.667 = \mathbf{0.044} \end{aligned}$$

$P(X|\text{buys} = \text{no})$

$$\begin{aligned} &= P(\text{age}=\text{youth}/\text{buys} = \text{no}) \times P(\text{income}=\text{medium}|\text{buys}=\text{no}) \times P(\text{student}=\text{yes}/\text{buys}=\text{no}) \\ &\times P(\text{credit_rating}=\text{fair}|\text{buys}=\text{no}) \\ &= 0.6 * 0.4 * 0.2 * 0.4 = \mathbf{0.019} \end{aligned}$$

Numerical

$$P(\text{buys}=\text{yes}|X)$$

$$=P(X|\text{buys}=\text{yes}) \times P(\text{buys}=\text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(\text{buys}=\text{no}|X) = P(X|\text{buys_computer}=\text{no}) \times P(\text{buys_computer}=\text{no})$$

$$0.019 \times 0.357 = 0.007$$

Since,

$$P(\text{buys} = \text{yes} | X) > P(\text{buys} = \text{no} | X),$$

**Therefore, the Naïve Bayesian Classifier prediction for Given X is: buys_computer
yes.**

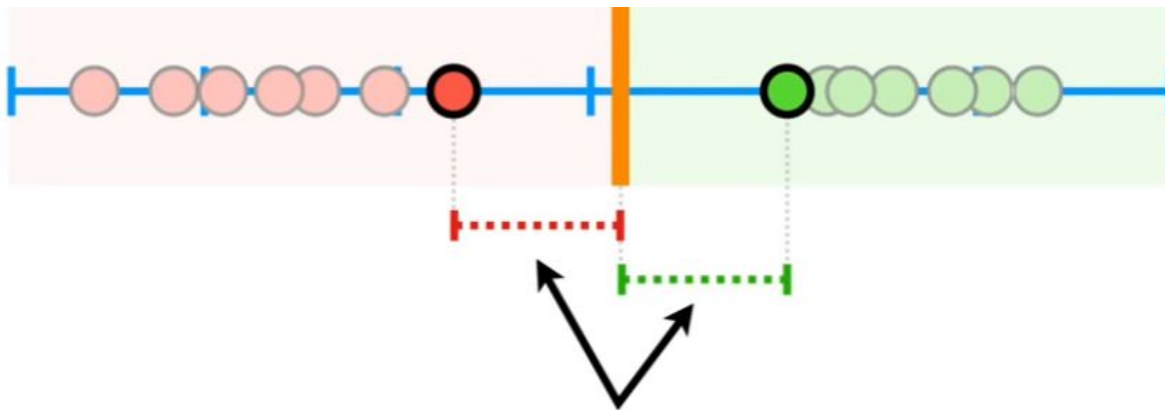
Support Vector Machine

- ❖ **Margin:** A decision boundary that can classify the unseen data.
- ❖ **Maximum Margin:** The widest possible street or clearance between the decision boundary (hyperplane) and the closest data points of any class (the support vectors).

$$M = \frac{2}{\|w\|}$$

where:

- w is the weight vector perpendicular (normal) to the separating hyperplane.
- $\|w\|$ is the Euclidean norm (length) of the weight vector, calculated as $\sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$.



Maximum Margin

❖ Hard Margin:

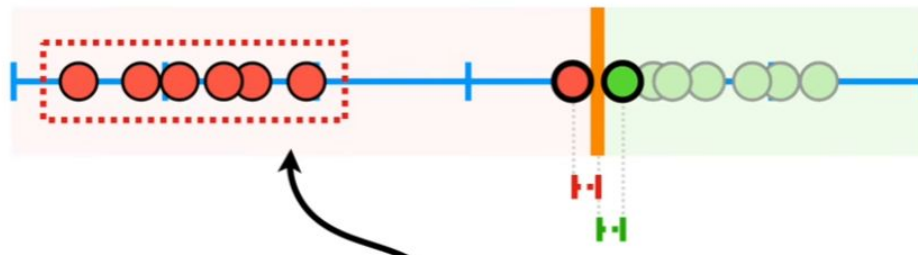
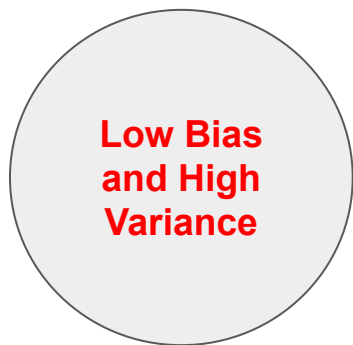
- The classical, strict formulation of a Support Vector Machine that requires zero classification errors and allows no data points inside the margin.

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

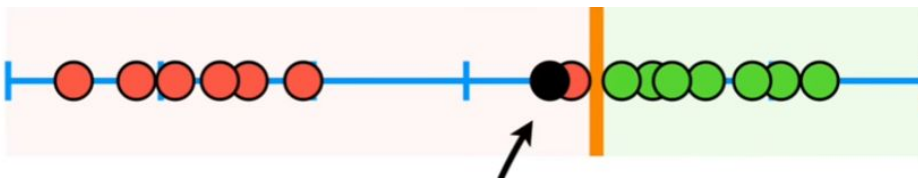
subject to $y_i(w^T x_i + b) \geq 1$ for all $i = 1, 2, \dots, N$

where:

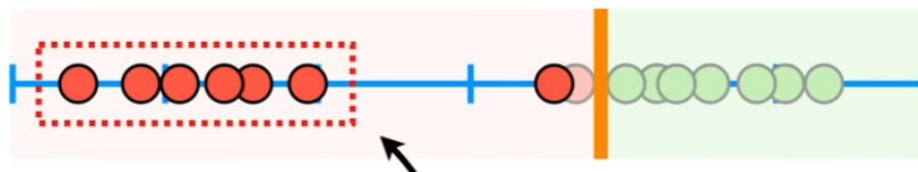
- x_i is the feature vector for instance i .
- $y_i \in \{-1, +1\}$ is the class label.
- w is the weight vector, and b is the bias term.



Max Margin



New Data to predict



Data are more
collected in another
side

Sensitive to
Outliers

❖ Soft Margin:

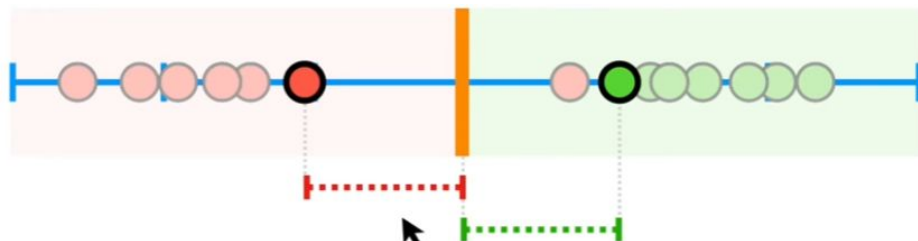
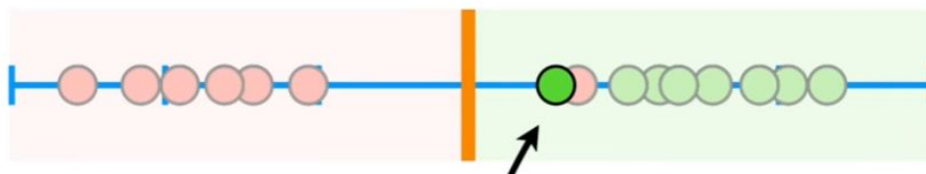
- To make the threshold less sensitive to outliers, Miss Classification must be allowed
- Bias/variance Tradeoff
- We use Cross validation to find soft margin,
 - Minimum Error as possible
- Support Vector: The specific data points that lie closest to the decision boundary
- Also called Support Vector Classifier.

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i$$

subject to $y_i(w^T x_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all $i = 1, \dots, N$

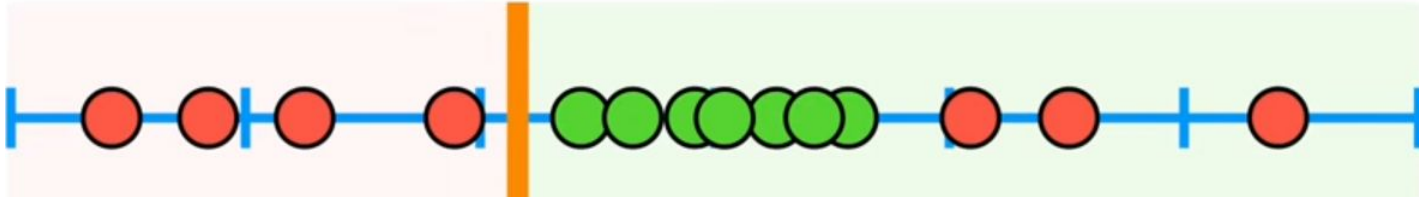
where:

- $\frac{1}{2} \|w\|^2$ controls the margin width.
- $\sum_{i=1}^N \xi_i$ sums up the total margin violations across all points.
- C is a hyperparameter that controls the trade-off between margin size and classification errors.



Support Vector

What if?



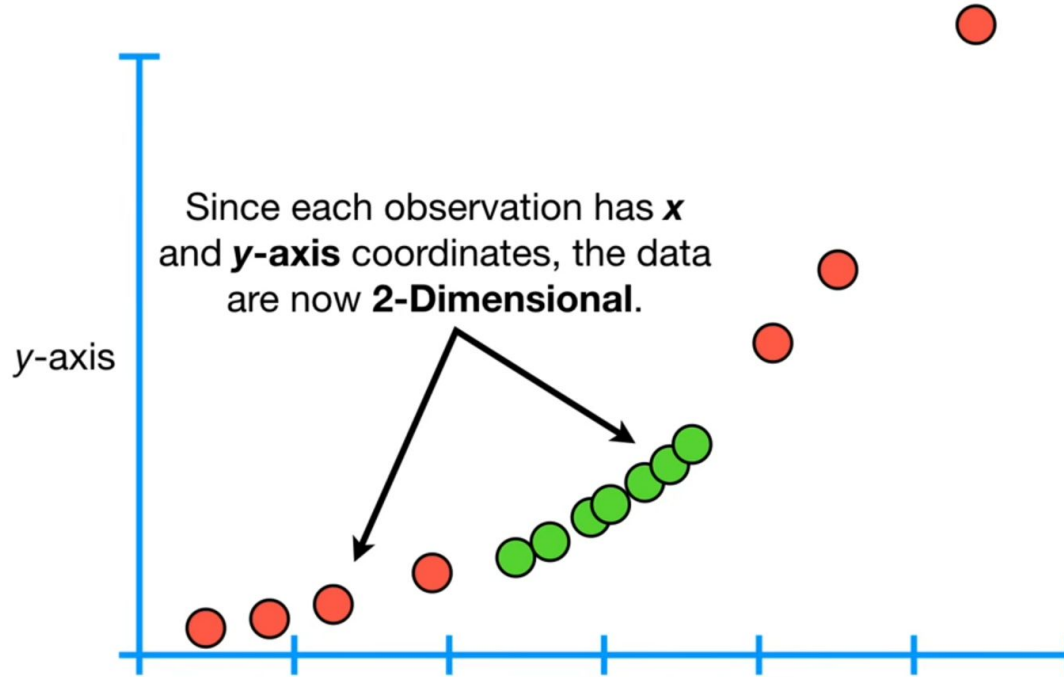
Maximal Margin Classifier



Support Vector Classifier



Where actual technique “Support Vector Machine” Introduced



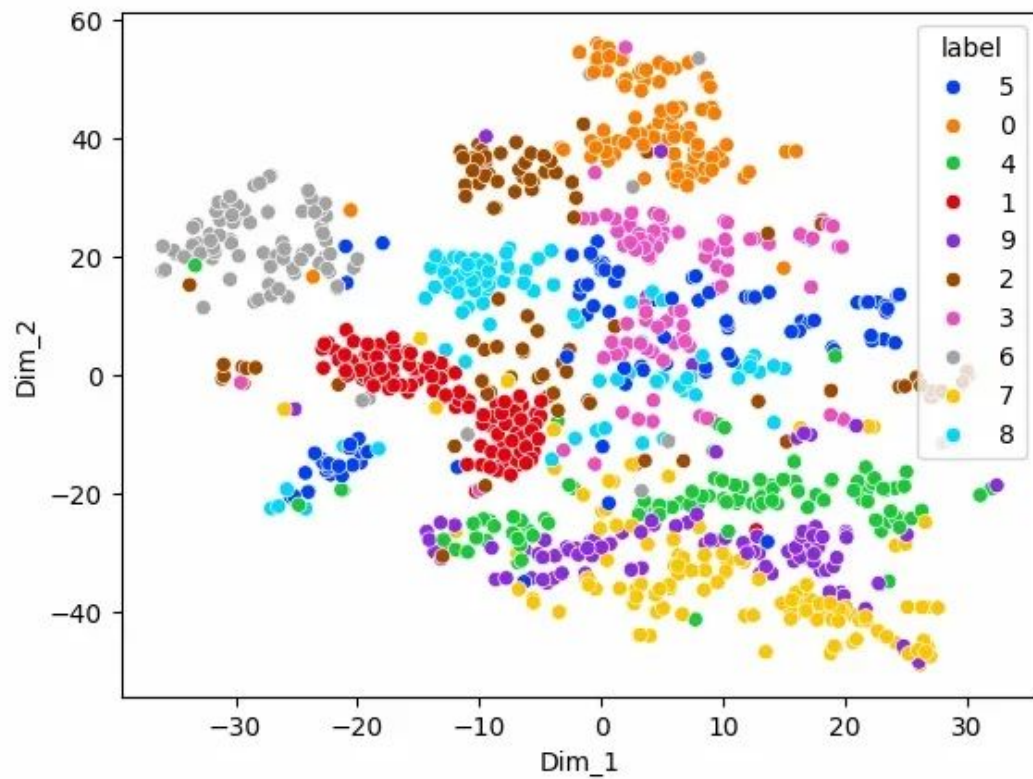
- Lower dimension to higher dimension
- Transfer of data is done with the Kernel function: Polynomial kernel, Radial Kernel
- Radial Kernel: works with the infinite dimension
- Kernel Trick: Think they are higher dimension but compare each data point

SVM Numerical

T-distributed Stochastic Neighbor Embedding (t-SNE)

- ❖ A non linear dimensionality reduction technique
- ❖ Visualizing high-dimensional data in a lower-dimensional space mainly in 2D or 3D
- ❖ t-SNE focus on preserving the local structure and pattern of the data.
- ❖ It helps in improving analysis and computational efficiency.
- ❖ t-SNE works by looking at the similarity between data points in the high-dimensional space.
- ❖ **Advantages:**
 - Great for Visualization: complex high-dimensional data into 2D or 3D for visualization
 - t-SNE focus on maintaining the local relationships between data points meaning similar data points remain close in the lower-dimensional space.
 - captures non-linear dependencies in the data which makes it suitable for complex datasets
- ❖ **Disadvantages:**
 - t-SNE is slower and more computationally expensive compared to linear methods especially on large datasets.
 - The output can vary with each run due to its randomness unless a fixed random_state is used.
 - It struggles with very large datasets (e.g., millions of points) unless optimized or approximated versions are used.

t-SNE



Reinforcement Learning (Q-Learning)

- ❖ Trial and Error Algorithm
- ❖ The goal of Q-learning is to learn the optimal action-selection policy for an agent interacting with an environment.
- ❖ $Q \rightarrow$ Quality, Lookup table (Q-table)
- ❖ Mathematically Equation,

$$Q(S, A) \rightarrow Q(S, A) + \alpha (R + \gamma (\max_{a'} Q(S', A')) - Q(S, A))$$

- $Q(s, a)$ is the Q-value for state s and action a .
- - α is the learning rate, controlling how much new information overrides old information.
- - R is the immediate reward for taking action a in state s .
- - γ is the discount factor, representing the importance of future rewards.
- - $\max_{a'} Q(s', a')$ is the maximum Q-value for the next state s' , representing the best possible reward achievable from that state.

Reinforcement Learning (Q-Learning)

S0	S1	S2	S3
S4	S5	S6	S7
S8	S9	S10	S11
S12	S13	S14	S15

- ❖ Start State: S0
- ❖ Goal State: S15
- ❖ Obstacles: S5 and S10
- ❖ Action: Up, Down, Left, Right
- ❖ Learning rate (α): 0.5
- ❖ Reward (R): +10 (Goal), -10(Obstacle), -1(Normal)
- ❖ Discount rate (γ): 0.9
- ❖ Formula:

$$Q(S, A) \rightarrow Q(S, A) + \alpha(R + \gamma(\max_a Q(S', A')) - Q(S, A))$$

Working Mechanism: Q-Learning

S	UP	Down	Left	Right
S0	0	0	0	0
S1	0	0	0	0
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
S15	0	0	0	0



Ep 1:

- S0 → S1
 - $Q(S0, R) \rightarrow 0 + 0.5(-1 + 0.9(0) - 0) = -0.5$
- S1 → S5
 - $Q(S1, R) \rightarrow 0 + 0.5(-10 + 0.9(0) - 0) = -5$
- **STOP**



EP 2:

- S0 → S4
 - $Q(S0, DOWN) \rightarrow 0 + 0.5(-1 + 0.9(0) - 0) = -0.5$
- **TO** S4 → S8, S12 → S13, S14 → S15 = -0.5
- S14 → S15
 - $Q(S14, RIGHT) \rightarrow 0 + 0.5(10 + 0.9(0) - 0) = 5$



Backpropagation



Ep 3

- Q(S14, RIGHT)
 - $Q(S13, RIGHT) = -0.5 + 0.5(-1 + 0.9 * 5 + 0.5) = 1.5$
- Repeat until Q(S0)
- Update the Q table

Updated Q-table

S	UP	Down	Left	Right
S0	1.21	5.54	1.21	-0.5
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
S14	4.20	8.72	6.72	9.73
S15	-5	8.58	8.58	10

Training Time

- ❖ The total time required for the model to learn from the training dataset until convergence.
- ❖ Floating Point Operations) to measure total work and FLOP/s (Floating Point Operations Per Second) to measure hardware speed.
- ❖ The total training time equals the total required FLOP divided by the speed of the GPU cluster (FLOP/s)

$$\text{Training Time} = t_{\text{end}} - t_{\text{start}}$$

- ❖ Why?
 - User experience
 - Real-time processing
 - Lower battery consumption
 - Higher throughput
 - Lower cloud inference cost

Training Energy

- ❖ Training large AI models uses FLOPs (Floating Point Operations), measuring total arithmetic calculations like additions and multiplications.
- ❖ Frontier models require massive compute budgets, ranging from 10^{25} to 10^{26} FLOPs, consuming gigawatt-scale electricity.

$$\text{Energy} = \text{Power} * \text{Time}$$

- ❖ Example:
 - GPU power 350 W; Training Time = 3 hours
 - Energy = $350 \times 3 = 1050 \text{ Wh} = 1.05 \text{ kWh}$
 - If electricity costs \$0.20/kWh
 - Training cost
 - $1.05 \times 0.20 = \$0.21$
 - Large foundation models may consume thousands to millions of kWh during training.

Model Size

- ❖ The parameter count and the physical memory/storage footprint (RAM or VRAM) required to store the model's weights.
- ❖ Usually determined by
 - Number of parameters
 - Precision of parameters

Model Size (in Bytes) = Total Parameters x Bytes per Parameter

- ❖ To convert the resulting bytes into Gigabytes (GB), divide by $1,024^3$ (or approximately 1.07×10^9):

Model Size (GB) = Total Parameters x Bytes per Parameter / 1024^3

Model Size

Example:

If you take an 8-billion-parameter model (8×10^9 parameters):

At 16-bit (FP16 / BF16):

$8,000,000,000 \times 2 \text{ bytes} = 16,000,000,000 \text{ bytes} \approx 14.90 \text{ GB}$

Inference time or energy

- ❖ The operational cost at runtime—specifically latency (time to first token / tokens per second) and energy consumed per user query.
- ❖ **OR** The duration a trained model takes to process an input prompt and generate a response.
- ❖ Active parameter count, serving batch size, Key-Value (KV) cache memory footprint, and sequence/prompt length.
- ❖ High latency hurts real-time user experiences in chatbots or autonomous driving.
- ❖ Advanced reasoning algorithms allow models to "think" longer at runtime, increasing the time spent per query to improve accuracy
- ❖ **Inference Energy**
 - The electrical power consumed by hardware (like GPUs) during query execution.
 - Standard frontier queries use roughly 0.31 Wh of energy, whereas long reasoning/agent tasks can multiply energy use by 13x or more.
 - Techniques like model quantization, pruning, and efficient batching aim to reduce the energy cost per token.

Cross Entropy

- ❖ In classification problems, a machine learning model predicts the probability of each class for any given input.
- ❖ Each data point truly belongs to only one class (probability 1 for one class, 0 for others).
- ❖ Cross-entropy loss is a way to measure how close a model's predictions are to the correct answers in classification problems.
- ❖ Types of Cross-Entropy Loss Function
 - Binary Cross Entropy Loss
 - Binary Cross-Entropy Loss is a widely used loss function in binary classification problems.

$$BCE = -\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(p_i) + (1 - y_i) \log(1 - p_i))$$

Cross Entropy

❖ Multiclass Cross Entropy Loss

- Multiclass Cross-Entropy Loss, also known as categorical cross-entropy or softmax loss is a widely used loss function for training models in multiclass classification problems.

$$CE = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C (y_{i,j} \cdot \log(p_{i,j}))$$

❖ Key Features:

- **Probabilistic Interpretation:** Guides models to output probabilities near the true class labels.
- **Differentiable:** Supports optimization via gradient descent.
- **Standard for Neural Networks:** Especially with softmax (multiclass) or sigmoid (binary) output layers.
- **Strong Penalization:** Assigns high penalty to confident but wrong predictions.
- **Library Support:** Implemented in all major ML libraries like PyTorch, TensorFlow, scikit-learn, etc.

Confusion Matrix

- ❖ Confusion matrix is a simple table used to measure how well a classification model is performing.
- ❖ It compares the predictions made by the model with the actual results and shows where the model was right or wrong.

Confusion Matrix

- ❖ TP: Model correctly predicted "Yes" (e.g., they bought the computer).
- ❖ TN: Model correctly predicted "No" (e.g., they didn't buy).
- ❖ FP (Type I Error): Model predicted "Yes," but it was actually "No."
- ❖ FN (Type II Error): Model predicted "No," but it was actually "Yes."

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Confusion Matrix

❖ Accuracy

- shows how many predictions the model got right out of all the predictions.
- It gives idea of overall performance but it can be misleading when one class is more dominant over the other.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

❖ Precision (Positive Predictive Value)

- Precision focus on the quality of the model's positive predictions.
- It tells us how many of the "positive" predictions were actually correct.
- Of all the times the model predicted "Yes," how often was it right?

$$\text{Precision} = \frac{TP}{TP+FP}$$

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Confusion Matrix

❖ Sensitivity / Recall (True Positive Rate)

- Of all the actual "Yes" cases, how many did the model find?
- Recall measures how good the model is at predicting positives.
- It shows the proportion of true positives detected out of all the actual positive instances.
- High recall is essential when missing positive cases has significant consequences like in medical tests.

$$\text{Recall} = \frac{TP}{TP+FN}$$

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Confusion Matrix

❖ Specificity (True Negative Rate)

- Of all the actual "No" cases, how many did the model correctly identify?
- Specificity is another important metric in the evaluation of classification models particularly in binary classification.
- It measures the ability of a model to correctly identify negative instances.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Confusion Matrix

❖ F-Score (F1-Measure)

- The harmonic mean of Precision and Recall. It is useful when you have an imbalanced dataset (e.g., 90% "Yes" and 10% "No").
- F1-score combines precision and recall into a single metric to balance their trade-off.
- It provides a better sense of a model's overall performance particularly for imbalanced datasets.
- It is helpful when both false positives and false negatives are important though it assumes precision and recall are equally important but in some situations one might matter more than the other.

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Maximum Likelihood Estimation (MLE) vs Maximum A Posteriori Estimation (MAP)

Aspect	MLE	MAP
Prior Used?	No	Yes
Objective	Maximize likelihood	Maximize posterior
Regularization	No implicit regularization	Implicit regularization via prior
Interpretation	Best fit to observed data	Best estimate given data and prior
Sensitivity to data	High (may overfit)	Lower (can reduce overfitting)